MANCHESTER 1824
The University of Manchester

A Modular Real-Time Haptic Rendering System Integrating Physics-Based Simulation and OpenGL Visualisation

A dissertation submitted to The University of Manchester for the degree of

Bachelor of Engineering in Mechatronic Engineering

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

11349510

School of Engineering

Contents

Abstract

This dissertation presents the design, implementation, and evaluation of a low-cost real-time haptic rendering system integrating a two-degree-of-freedom physical interface with a custom simulation and visualisation framework. The approximately £180 hardware platform is developed as an accessible alternative to commercial haptic devices while addressing the challenge of bounded bidirectional haptic interaction under practical constraints, including limited actuator capability and non-negligible communication latency.

The system combines embedded torque-controlled actuation, high-rate host–device communication, physics-based simulation, and proxy-based haptic rendering within a multi-threaded software architecture. The main contributions are the co-design of the complete closed-loop pipeline, quantitative separation of host-side processing from transport delay, and characterisation of USB CDC burst behaviour together with a snapshot-based mitigation strategy.

Experimental evaluation demonstrates nominal 1 kHz embedded packet generation and host-side haptic/device-loop execution with low host-side processing latency (mean 0.299 ms, 99th percentile 2.04 ms). However, this should not be interpreted as a true 1 kHz end-to-end haptic loop, since force computation operated on latest-available state snapshots affected by asynchronous simulation timing and a consistent communication round-trip delay of 15.60–15.67 ms. Simulation validation produced a fitted virtual stiffness of 489.57 N m^{-1} with $R^2 = 0.9973$, showing that the command-side force model reproduced the configured 500 N m^{-1} coupling stiffness. Despite the communication delay, bounded and controllable contact behaviour was observed during the tested interactions, supported by snapshot-based communication, virtual coupling with damping, actuator-side torque rate limiting at 55 N m s^{-1}, force saturation at 31 N, and watchdog fail-safe behaviour.

The results show that communication latency, rather than computational performance, is the dominant factor limiting achievable haptic fidelity, imposing a practical bound on renderable virtual stiffness. Hardware limitations, including the absence of current sensing and transmission non-idealities, further constrain force accuracy and consistency.

Overall, the work demonstrates bounded and usable real-time haptic interaction on a low-cost platform, while highlighting the central role of communication architecture and control design in determining system performance. Final hardware validation was limited to single-axis force feedback due to actuator failure, so full two-degree-of-freedom force-rendering conclusions remain provi-

sional. The findings provide insight into the trade-offs between cost, contact behaviour, responsiveness, and fidelity in practical haptic systems, and identify clear directions for future improvement.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on Presentation of Theses.

Acknowledgments

Use of AI Tools

Large language models (including ChatGPT) were used during this project as a supplementary tool to assist with language refinement, report structuring, and code debugging.

These tools were used to improve clarity, grammar, and organisation of written content, and to support troubleshooting during software development.

All technical content, system design, implementation decisions, analysis, and conclusions are my own work. All material included in this report has been reviewed and verified by me.

1 Introduction

This section introduces the motivation for the project, defines the problem being addressed, and sets out the measurable objectives used to evaluate whether the work was successful. It also summarises the main contributions and the practical scope limitations of the final prototype.

1.1 Background and Motivation

Robotic manipulators are increasingly used in domains such as industrial automation, teleoperation, and robot-assisted surgery, where safe and effective interaction with physical environments is essential. Although visual feedback provides important global information about the workspace, it alone may be insufficient for developing fine motor control, accurate force regulation, and intuitive understanding of contact conditions.

Haptic interfaces address this limitation by enabling bidirectional physical interaction between a user and a virtual or remote environment. By rendering forces, such systems can communicate contact, constraint, and interaction information that is difficult to convey visually alone [1]. In robotic manipulation tasks, this is particularly valuable for training applications, where users must learn not only where to move, but also how to regulate force and respond to contact conditions. Prior work has shown that haptic feedback can improve task performance and reduce excessive applied force, particularly for less experienced users [2].

Despite these benefits, access to haptic systems remains limited in research and educational settings. Commercial interfaces provide mature and high-performance force feedback, but are typically priced substantially above academic budgets and are often distributed through vendor-specific SDK integration pathways [1]. This can restrict accessibility and make it difficult for researchers and students to experiment with alternative control strategies, custom simulation environments, or modified hardware configurations. As a result, there is strong practical motivation for low-cost and modular haptic platforms that are suitable for prototyping, experimentation, and robotics training research.

At the same time, achieving stable and convincing haptic interaction is technically challenging. Real-time haptic rendering commonly requires update rates on the order of 1 kHz together with low latency and carefully designed control structure in order to render contact stably and responsively [3]. The stability of haptic interaction is particularly sensitive to delay in the feedback loop, since

communication and computation delays introduce phase lag and reduce the maximum achievable virtual stiffness [4]. Consequently, the design of a practical haptic system is not simply a matter of adding actuators to an input device: it requires coordinated consideration of mechanical design, embedded control, communication architecture, simulation coupling, and force rendering strategy.

1.2 Problem Definition

This dissertation addresses the design and evaluation of a low-cost real-time haptic interface system for robotic manipulation training, with explicit consideration of practical implementation constraints. The specific challenge is not merely to construct a haptic input device, but to integrate multiple subsystems into a bounded bidirectional interaction loop: user motion must be sensed at the device, transmitted to a host system, interpreted within a simulated environment, converted into interaction forces, mapped back into actuator commands, and rendered to the user in real time. In practice, each stage of this loop introduces constraints. Limited actuator capability affects the achievable force output, communication delay limits responsiveness and achievable virtual stiffness, and software architecture determines whether the system remains extensible and maintainable.

Existing work demonstrates the value of haptic feedback and the feasibility of low-cost haptic hardware, but there remains a practical gap between simple hardware demonstrations and fully integrated systems that combine low-cost actuation, real-time communication, physics-based simulation, and explicit treatment of latency and stability. Educational and open-source devices have shown that affordable haptic hardware is possible [5], [6], and low-cost 2-DOF academic systems have also been demonstrated [7]. However, comparatively less emphasis is placed on complete end-to-end architectures in which hardware, embedded control, host-side simulation, rendering, and communication are developed together and evaluated as an integrated real-time system. Communication architecture is therefore a central design constraint in this project, since buffering, scheduling, and transport delay can directly affect both stability and contact fidelity.

The problem addressed in this project is therefore defined as follows: *to design, implement, and evaluate a low-cost haptic rendering system that is sufficiently modular for experimentation, sufficiently responsive for real-time interaction, and capable of bounded, controllable bidirectional haptic coupling with a simulated robotic environment*.

1.3 Aim and Scope of This Dissertation

The project focuses on demonstrating the feasibility of a complete end-to-end haptic architecture, covering: device design and construction; embedded firmware development for sensing, communication, and actuator control; implementation of a host-side framework integrating physics simulation, haptic rendering, and visualisation; and experimental evaluation of communication performance, timing, bounded contact behaviour, and practical interaction quality. The scope is intentionally limited. The dissertation does not attempt fully calibrated force output, commercial-level rendering fidelity, or a formal user study. Final hardware validation was performed with force feedback on one joint only, due to actuator failure during development. Consequently, conclusions about full 2-DOF force rendering are provisional, while the communication, host-side timing, simulation, and safety findings remain directly applicable because they do not depend on two-axis actuation. The principal contribution therefore lies in *system-level design*, *integration*, *and evaluation*, together with analysis of the practical constraints that limit achievable performance. As the results demonstrate, the dominant limitation is communication latency rather than host-side computation, and addressing this is identified as the primary route to higher-fidelity interaction.

1.4 Objectives

To achieve this aim, the project pursued six SMART objectives:

(i) Build a low-cost physical haptic interface with a total hardware cost below £200, capable of measuring joint position and accepting actuator torque commands during closed-loop operation.

(ii) Evaluate whether host–device packet streaming can be sustained at a nominal 1 kHz rate, while separately quantifying packet integrity, burst behaviour, and round-trip latency to determine the effective closed-loop timing limitation.

(iii) Demonstrate that host-side computation is not the dominant timing bottleneck by measuring mean closed-loop host processing latency below 1 ms during integrated operation.

(iv) Validate the host-side rendered contact model quantitatively using logged rendered-force data, showing a near-linear force–penetration relationship with fitted stiffness within 5% of the configured virtual coupling value and high goodness of fit.

(v) Verify safety behaviour by measuring force saturation, embedded torque limiting, torque slew-rate limiting, and watchdog torque removal during communication interruption.

(vi) Evaluate physical interaction performance under motor-enabled contact and constrained-motion tests, reporting penetration bounds, rendered-force command consistency, and limitations affecting generalisation to full multi-axis force feedback.

1.5 Contributions of This Work

Prior work on low-cost haptic platforms focuses primarily on hardware design [5], [6], [7] or on isolated software demonstrations, with comparatively little emphasis on fully integrated real-time systems in which hardware, firmware, communication, simulation, and rendering are co-developed and evaluated together. This work addresses that gap. Its principal contributions are:

- **A complete end-to-end haptic rendering system.** Custom hardware, embedded firmware, a packet-based communication protocol, and a modular host-side framework with real-time physics and OpenGL visualisation are integrated into a single bidirectional pipeline and evaluated as a unified system. This type of co-design and cross-layer evaluation is less commonly emphasised in existing low-cost academic haptic prototypes.
- **Demonstration of bounded contact interaction under non-negligible communication delay.** Bounded, controllable contact behaviour was observed during the tested interactions despite a measured round-trip latency of approximately 15–16 ms — more than an order of magnitude above the nominal 1 ms local haptic/device-loop period. This behaviour was achieved without a formal passivity guarantee through the combined use of virtual coupling, damping, snapshot-based communication, multi-threaded decoupling, and an embedded torque slew-rate limiter.
- **Quantitative identification of transport delay, not computation, as the dominant performance constraint.** Instrumented runtime logging isolates host-side processing latency from transport-layer delay (as quantified in Section 5.1.3), demonstrating that the communication interface, not the host processor, is the bottleneck. This finding directly motivates communication-architecture improvements as the primary route to higher haptic stiffness in this class of system.
- **Characterisation of USB CDC burst behaviour and a snapshot-based mitigation strategy.** Measured host-side packet inter-arrival times reveal strongly bimodal delivery: most packets arrive in near-simultaneous bursts separated by gaps of 14–16 ms, consistent with USB CDC buffering. A snapshot-based channel design is shown to decouple the haptic rendering loop from this irregular delivery pattern, preventing stale-state accumulation without requiring changes

to the underlying communication stack.

2 Literature review

This section reviews the literature needed to justify the system requirements and evaluation approach. It covers haptic interfaces for robotics, rendering and contact methods, stability under delay, communication timing, and low-cost hardware platforms.

2.1 Haptic Interfaces for Robotics

As established in Section 1.1, haptic interfaces provide force feedback that complements visual information and supports fine motor control in robotic manipulation and training contexts. This section reviews the relevant literature on rendering techniques, stability analysis, and low-cost hardware platforms.

Commercial haptic devices such as the 3D Systems *Geomagic Touch* and *Touch X*, the *Force Dimension* product range, and the Haply *Inverse3 X* are widely used in research, training, and simulation contexts [1]. These systems provide kinesthetic force feedback and are typically distributed with proprietary SDK integration workflows, although some also expose lower-level APIs for custom software development.

However, they are generally sold at substantially higher price points than low-cost academic prototypes. For example, the Geomagic Touch and Touch X are publicly listed at approximately US$3,400 and US$13,400 respectively [8], [9], while other systems are commonly distributed via quotation-based sales channels [10], [11]. In contrast, the system developed in this work has a total hardware cost of around £180, highlighting the potential for low-cost systems to provide accessible haptic functionality for research and training applications.

2.2 Haptic Rendering Techniques

Real-time haptic rendering has been extensively studied, with particular focus on contact-based interaction models. Two widely adopted approaches are virtual coupling and the virtual proxy (or “god-object”) method introduced by Zilles and Salisbury and later extended by Ruspini *et al.* [3], [12].

In these approaches, the device state is coupled to a constrained proxy within the virtual environment, enabling stable rendering of contact and geometric constraints. Reliable performance typically requires high local update rates on the order of 1 kHz, together with accurate tracking of device state and consistent mapping from Cartesian interaction forces to actuator torques [3]. This requirement applies to the freshness of the local haptic contact model, not only to nominal software loop frequency or packet throughput. In robotic implementations, these mappings are commonly derived using the manipulator Jacobian within operational-space control frameworks [13].

Haptic systems are commonly classified as impedance-type or admittance-type devices, depending on whether the device measures position and outputs force (impedance) or measures force and outputs motion (admittance). Most grounded haptic interfaces, including the system developed in this work, operate as impedance devices due to the relative ease of position sensing and actuation.

Contact detection must also be computationally simple enough for the haptic servo loop. Signed distance fields (SDFs) provide direct access to penetration depth and surface normal information from the sign and gradient of the distance function [14]. Compared with explicit mesh-contact tests or broad-phase bounding-volume methods alone, this representation is well suited to a high-rate proxy-based loop because the contact query produces the quantities required for both proxy projection and normal-force calculation.

The literature also motivates separating high-rate local haptic contact handling from slower application and graphical layers. Ruspini *et al.* describe a multilevel haptic rendering framework for complex graphical environments, while constraint-based proxy methods maintain a local contact point or proxy state that is updated continuously subject to contact constraints [3], [12], [15]. This supports the architectural distinction between a high-frequency local contact model and slower subsystems responsible for broader scene definition, visualisation, and application-level updates.

2.3 Stability in Haptic Systems

Stable haptic interaction imposes strict requirements on latency, control architecture, and numerical implementation. Adams and Hannaford demonstrated that virtual coupling can provide stability guarantees by decoupling device dynamics from the virtual environment under passive interaction assumptions [4].

More generally, stability in haptic systems is often analysed using passivity theory. Colgate and Schenkel showed that discrete-time haptic interfaces can become unstable when energy is artificially gen-

erated within the feedback loop due to sampling, delay, or numerical approximation errors [16]. Their passivity framework establishes conditions under which a haptic system remains energetically passive, thereby ensuring stable interaction with arbitrary passive users and environments.

These results highlight the sensitivity of haptic systems to delay and discrete-time effects. In practical implementations, communication latency and asynchronous computation can result in effective energy injection into the feedback loop, reducing stability margins and limiting the achievable virtual stiffness. Passivity-based analyses therefore motivate treating loop delay as a direct limit on renderable stiffness: as delay increases, the range of stable virtual coupling parameters becomes more restricted. This is particularly relevant in systems where communication delay is non-negligible relative to the control timestep, motivating the use of control structures that explicitly manage dynamic coupling, damping, and energy injection in order to maintain stable interaction.

2.4 Communication Timing in Real-Time Haptic Loops

Haptic rendering places unusually strict requirements on communication timing because the device state and returned force command form part of the same feedback loop. Prior work on sampled-data haptic interaction shows that delay and discretisation reduce stability margins [4], [16], while USB latency studies show that USB-based real-time control links can introduce delays on the order of milliseconds [17]. Interfaces that are convenient for prototyping, such as USB virtual COM ports using the USB CDC class, can therefore become a limiting factor when packets are buffered by the interface firmware, host driver, or operating system scheduler. Even when the embedded controller transmits samples at a regular rate, non-uniform host-side delivery can increase effective feedback delay if the control loop processes stale queued states.

For this reason, communication design should be considered alongside control design in practical haptic systems. A latest-value, or snapshot-based, transfer pattern is one suitable architectural response: the haptic loop consumes the newest available state rather than draining a backlog of historical samples. This does not remove transport delay, but it can prevent buffered packets from accumulating into additional application-level latency.

2.5 Low-Cost Haptic Hardware

Advances in low-cost components, including 3D-printed structures, microcontrollers, and magnetic encoders, have enabled the development of accessible haptic devices using widely available hardware. This has led to increasing interest in open and modular platforms for research and education [1].

The *Hapkit* platform developed by Martinez *et al.* demonstrates that functional haptic devices can be realised at very low cost (approximately US$50) using 3D-printed components [5]. Subsequent work extended this approach to modular multi-degree-of-freedom configurations, including both pantograph and serial mechanisms [6].

Similarly, Lavatelli *et al.* presented an open-source low-cost 2-DOF haptic device, demonstrating that multi-degree-of-freedom actuation can be achieved within an academic budget while maintaining useful interaction capability [7].

However, much of this work focuses primarily on hardware design or educational platforms. Comparatively less emphasis is placed on fully integrated systems that combine low-cost hardware with real-time communication, simulation coupling, and extensible software architectures.

2.6 Summary of Literature

The literature establishes that haptic feedback improves performance in robotic manipulation tasks and provides well-developed methods for stable force rendering, particularly through virtual coupling and passivity-based control frameworks. Low-cost hardware platforms have also demonstrated that accessible haptic devices are feasible.

However, there remains a specific gap between simplified hardware demonstrations and fully integrated real-time systems in which communication architecture, stability strategy, simulation coupling, and force rendering are evaluated together. This gap directly motivates the co-designed system architecture presented in Section 3.

3 System Design and Implementation

This section describes the realised system design, covering the physical device, embedded control and communication, and the host-side software stack used for rendering, simulation, and force feedback.

The system prioritises bounded real-time interaction, modularity, and reproducibility within the constraints of a low-cost prototype. Because haptic interaction is sensitive to update rate and end-to-end latency (Section 2), time-critical embedded control is separated from asynchronous host-side simulation, rendering, and logging.

3.1 System Overview

The proposed system implements a bidirectional haptic interaction loop consisting of four principal layers: the haptic device hardware, embedded control firmware, host-side simulation and haptic rendering, and graphical visualisation. A conceptual overview of the signal and force flow is shown in Fig. 1.

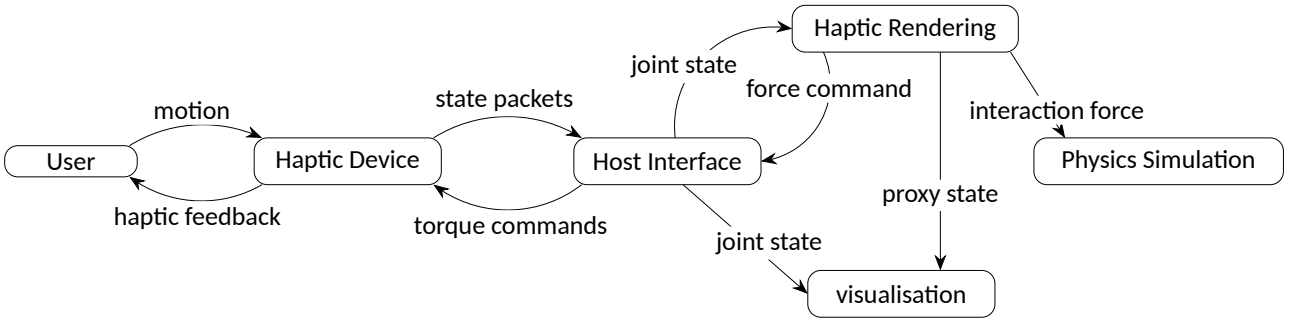


Fig. 1. High-level architecture of the proposed haptic system. User motion is measured by the haptic device and transmitted to the host interface. The host distributes state information to the haptic rendering and visualisation subsystems, while the haptic rendering module computes interaction forces, updates the simulation, and generates feedback commands that are returned to the device as actuator torque commands.

Time-critical control tasks are executed on the embedded system at high frequency, while simulation, rendering, and visualisation are performed asynchronously on the host.

3.2 Mechanical and Embedded Platform

The physical interface is a planar two degree-of-freedom serial mechanism designed for low-cost fabrication and straightforward maintenance. A two-link layout was selected to provide a usable

workspace while keeping actuator count, wiring complexity, and control overhead manageable.

The structure is primarily 3D printed, with each joint supported by steel shafts and rolling element bearings to reduce friction and improve backdrivability. Actuation is provided by brushless gimbal motors (T-Motor GB2208)[18]. The base joint incorporates a 3:1 GT2 belt reduction stage to increase available output torque, while the elbow joint is direct-drive to minimise mechanical complexity.

An adjustable idler pulley is integrated into the belt stage to enable tensioning and reduce backlash. Despite this, non-ideal behaviour such as compliance and occasional slip under high load was observed, as discussed in Section 6.4.

To ensure accurate joint angle measurement, absolute magnetic encoders (AS5048)[19] are mounted directly on the joint axes and interfaced via SPI. This is particularly important for the belt-driven joint, where transmission compliance would otherwise introduce discrepancies between motor-side and output angles.

For the belt-driven actuator, an additional AS5600 magnetic encoder[20] is mounted on the motor shaft and interfaced via I^2C. This provides rotor position feedback required for field-oriented control (FOC), while the joint encoder provides accurate output angle measurement.

A complete mechanical assembly, including component layout and link geometry, is provided in Appendix E. The assembled prototype is shown in Fig. 2, illustrating the realised mechanical structure and actuator configuration.

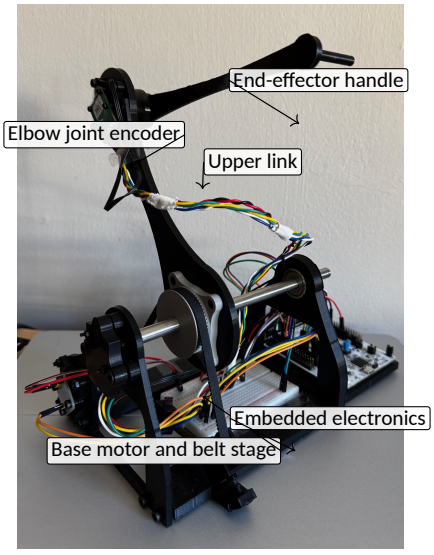


Fig. 2. Final assembled 2-DOF haptic interface prototype, annotated with the main mechanical and embedded components.

Embedded control runs on an STM32 Nucleo-G431RB[21] platform with SimpleFOC Mini drivers and the SimpleFOC software stack [22]. Both actuators operate in torque-mode FOC [23]. A communication watchdog is implemented in firmware: if valid host commands are not received within the timeout window, output torques are forced to zero. Additionally, a torque slew-rate limiter is applied to each actuator command, constraining the rate of change of commanded torque to reduce high-frequency oscillations during rapid contact transitions.

Due to hardware constraints, no direct force sensing or current measurement was implemented; as a result, torque output is estimated rather than directly measured. Additionally, one actuator failed during development, meaning final hardware validation was conducted using single-axis actuation, although both joints remained available for sensing.

3.3 Communication Protocol and Device Interface

Host–device communication uses fixed-format binary packets over a USB virtual serial link at a nominal 1 kHz packet streaming rate. Device state packets contain a sequence number, MCU timestamp, and joint angles; torque command packets contain the command sequence, referenced state sequence, and commanded joint torques. Sequence numbers support drop and ordering checks, while timestamps support timing analysis. Both packet types include a two-byte header for stream re-synchronisation and a checksum for integrity checks. The target packet rate defines the intended communication cadence, but not the end-to-end freshness of the closed-loop haptic state.

On the host, serial I/O and protocol handling are split across two layers: *SerialLink* providing a low-level COM read/write abstraction, and *DeviceAdapter* handling packet parsing, forward kinematics, Cartesian force-to-joint torque mapping, and message-bus integration.

The device adapter runs in a dedicated nominal 1 kHz loop. At each cycle, it retains the newest valid state packet, publishes tool pose and velocity to the haptic engine, consumes the latest wrench command, computes joint torques using the planar Jacobian transpose relation $\tau = J^T F$, and transmits the resulting torque command packet.

This ensures that control decisions are based on the most recent state available to the host, avoiding backlog-induced latency and improving responsiveness under high update rates. However, the newest available packet may still be delayed by the transport layer, so this mechanism mitigates queued stale samples rather than eliminating communication latency.

3.4 Host Software Architecture

The host application is implemented as a modular multi-threaded C++ system. Major responsibilities are partitioned into the renderer, world manager, physics engine, haptic engine, and device adapter.

Inter-thread communication is implemented through a message bus with two channel types: queue channels for discrete commands and events (e.g., world edits, wrench messages), and snapshot channels for latest-value shared state (e.g., world snapshots).

This separation allows command traffic to be drained in batches while continuous state can be accessed with low latency. Snapshot channels are used specifically to decouple the haptic loop from non-uniform packet delivery: when serial packets arrive in bursts, the loop consumes the newest state rather than accumulating stale samples in the feedback path.

3.4.1 World and Rendering Subsystems

WorldManager is the authoritative owner of virtual scene state. Objects store identity, geometry reference, pose, appearance, role labels, and physics properties. Subsystems do not mutate world data directly; instead, they publish commands which are applied centrally by the world manager.

Rendering is handled by an OpenGL subsystem running on the main thread. Each frame reads the latest world snapshot, resolves geometry and mesh handles, composes model transforms from object pose, and renders via shared shader and mesh abstractions. An ImGui interface provides runtime controls for object editing and parameter tuning; UI edits are emitted as world commands to preserve ownership boundaries.

3.4.2 Haptic Rendering Engine

The haptic engine executes at a nominal 1 kHz in a dedicated loop. At each cycle, it reads the latest world snapshot and newest tool state, performs contact queries, computes force output, and publishes results to device, physics, and visualisation channels. This rate refers to the local haptic rendering computation cadence. It does not imply that the world state, device state, or returned force feedback are refreshed end-to-end every 1 ms, since the loop operates on latest-available snapshots that may be older due to communication latency and asynchronous simulation timing.

Contact detection is performed using signed distance field (SDF) queries, with penetration depth and surface normals obtained from the local distance field gradient [14]. For each candidate object, the tool position is transformed into object-local coordinates and queried against that object's SDF. If penetration is detected (negative signed distance), the tool point is projected to the surface to define the proxy position; otherwise, the proxy follows the tool in free space.

Force output is generated using a proxy-based virtual coupling model, in which a spring–damper relationship is defined between the device position and a contact-constrained proxy [4]:

$$\mathbf{F} = K\left(\mathbf{x}_p - \mathbf{x}_t\right) + D\left(\mathbf{v}_p - \mathbf{v}_t\right)$$

where $\mathbf{x}_p, \mathbf{x}_t$ are proxy and tool positions and $\mathbf{v}_p, \mathbf{v}_t$ are corresponding velocities. In the implemented configuration, $K = 500\,\mathrm{N\,m^{-1}}$, $M = 0.02\,\mathrm{kg}$, and $D = 0.7 \cdot 2\sqrt{KM}$. The stiffness was selected as a conservative value for the measured communication delay while still producing clearly perceptible contact forces; the fitted stiffness reported in Section 5.2.2 and the observed contact behaviour discussed in Section 6.3 are used to evaluate this choice.

This follows the standard virtual coupling formulation, although the present implementation does not explicitly enforce passivity-based design constraints. In particular, the proxy state is recomputed directly from the current tool position and contact geometry at each timestep, rather than being evolved as a persistent constrained dynamic state. As a result, the implementation captures the force-generation behaviour of virtual coupling but does not realise the full passivity-based formulation described by Adams and Hannaford, nor the continuously evolved contact proxy used in constraint-based haptic rendering approaches [3], [12], [15].

Accordingly, the force command is generated using a deliberately conservative coupling configuration, with damping and actuator-side limiting used to reduce abrupt force changes during contact transitions. The observed behaviour of this design is discussed in Section 6.3.

To maintain safe interaction, force magnitude is clamped to a maximum of 31 N before publication.

3.4.3 Physics Simulation Integration

Rigid-body dynamics are simulated using NVIDIA PhysX. The physics subsystem runs on its own simulation thread and consumes wrench messages from the haptic loop. Incoming forces are converted to impulses $\mathbf{J} = \mathbf{F}\,\Delta t$ over the effective timestep and applied at the contact point to the

corresponding actor.

A fixed-step accumulator scheme is used to decouple simulation stability from variable external loop timing. When world topology or physics properties change, actors are rebuilt from the latest authoritative world snapshot. After each simulation update, dynamic poses are written back to the world manager so subsequent rendering and haptic queries operate on consistent state. In the realised architecture, the physics engine therefore provides the authoritative global scene evolution at its own rate, while the haptic engine consumes snapshots of that state rather than independently evolving a local dynamic model of contacted objects.

3.5 Logging and Communication Validation

To avoid introducing disk latency into the 1 kHz control path, runtime logging is performed asynchronously. The device thread publishes structured timing and state messages to dedicated logging channels, and a background logger thread drains these channels into memory.

At shutdown, logs are exported to `device_timing.csv` for matched state-command timing records and `device_state_log.csv` for every valid parsed state packet. The timing log captures the packet parsing, tool publication, wrench consumption, and torque transmission timestamps used in Section 5.1.3.

A separate `serial_tests` executable characterises serial performance independently of the full application. It supports packet rate/integrity testing and ping–echo round-trip latency testing, using the same buffered parser, header search, checksum validation, and re-synchronisation approach as the main device interface.

4 Experimental Methodology

This section defines the test procedures used to evaluate communication performance, closed-loop timing behaviour, simulation fidelity, safety behaviour, and hardware interaction characteristics of the proposed haptic rendering system. The section focuses on what was tested and how measurements were collected; quantitative outcomes are reported in Section 5 and interpreted in Section 6.

4.1 Communication Performance Tests

Communication performance was characterised using the dedicated `serial_tests` utility described in Section 3.5. Its rate/integrity and ping–echo modes export CSV files for offline statistical analysis.

Tests were conducted across target transmission rates from 100 Hz to 2000 Hz to assess the effect of increasing packet frequency on reliability and timing performance.

4.1.1 Packet Integrity and Rate Tests

Packet integrity and communication timing were evaluated using the rate-test mode of the communication test utility.

For each target transmission frequency, the firmware was configured to stream fixed-format device state packets at a constant rate. Tests were performed at nominal frequencies of 100 Hz, 250 Hz, 500 Hz, 1000 Hz, 1250 Hz, and 2000 Hz.

For each test condition, the host continuously received and parsed incoming packets over the USB virtual COM interface. The recorded performance metrics included valid packet count, dropped packets inferred from sequence gaps, out-of-order packets, host-observed inter-arrival time, effective receive rate, and inter-arrival jitter.

4.1.2 Round-Trip Latency Test

Round-trip latency was evaluated using a ping–echo communication test.

In this test, the host transmitted timestamped ping packets to the embedded controller, which immediately returned an echo packet containing the original sequence number and timestamp.

Tests were performed using transmission intervals of 1 ms, 2 ms, and 10 ms. For each test, the evaluated metrics included mean, median, 95th percentile, and 99th percentile RTT, along with minimum and maximum RTT values and timeout counts.

These results were used to characterise communication latency and timing consistency under different transmission loads.

4.1.3 End-to-End Host-Side Timing Test

In addition to the standalone communication tests, end-to-end timing of the host-side haptic pipeline was evaluated using the runtime logging framework described in Section 3.5. Logs captured during normal closed-loop operation were analysed in MATLAB to compute host-side latency, intermediate pipeline delays, matched state-command consistency, parsed state-packet inter-arrival timing, sequence gaps, parsed packet rate, and MCU-side timing trends.

Because the host and microcontroller clocks were not synchronised, the analysis was restricted to host-side timing differences and trend-based interpretation of MCU timestamps. Absolute one-way latency between host and device could therefore not be inferred directly.

4.2 Simulation Validation

Simulation validation tests were designed to assess the command-side behaviour of the haptic rendering model independently of physical actuator dynamics.

The simulation tests focused on planar virtual-wall behaviour. A single-depth hold test assessed whether the rendered force command remained steady during sustained penetration. A multi-depth hold test quantified the force–penetration relationship by holding the end-effector at several penetration depths and fitting the logged rendered-force data. Additional free-space and deep-penetration checks verified that the renderer produced negligible force outside contact and that host-side force saturation was applied under extreme penetration. Physical closed-loop interaction with actuator feedback was evaluated separately through the repeated-contact and constrained-motion hardware tests in Sections 4.4.1 and 4.4.2.

4.2.1 Virtual Wall Test

A virtual wall test validated unilateral contact behaviour by moving the end-effector toward and against a planar boundary. The evaluation examined whether force was near zero outside contact, remained steady during a single-depth hold, increased approximately linearly with penetration depth, and matched the implemented spring–damper coupling model while the proxy remained constrained to the wall.

4.2.2 Free-Space and Saturation Checks

Free-space and deep-penetration checks were used to verify the boundary cases of the renderer. The free-space check confirmed that no significant force was generated outside contact. The deep-penetration check verified that large virtual penetrations were limited by the configured host-side force clamp before torque commands were generated.

4.3 Safety Validation

Safety validation tests evaluated whether the system remained bounded and behaved predictably under both normal and fault conditions. These tests focused on checking torque saturation, rate limiting, and watchdog-based fail-safe behaviour. The reported safety metrics were saturation fraction, maximum raw and clamped torque, applied torque cap, measured torque slew-rate percentiles, watchdog activation count, total watchdog-active time, activation duration, and sequence-matched command consistency.

4.3.1 Torque Saturation Test

A torque saturation test verified that host-side force limits and embedded torque clamps kept actuator commands within safe bounds during high-force contact.

4.3.2 Rate Limiting Test

A rate limiting test evaluated the effect of constraining the rate of change of commanded torque at the embedded controller level.

Testing verified that the embedded slew-rate limiter smoothed rapid command changes into bounded torque ramps and reduced high-frequency oscillation during contact transitions.

4.3.3 Watchdog Timeout Test

A watchdog timeout test verified safe system behaviour under communication loss by interrupting the incoming command stream during operation.

The test verified that the watchdog triggered when command updates ceased, removed commanded torque within the timeout window, and returned the device to a zero-torque command state.

4.4 Hardware Validation

Hardware validation assessed the behaviour of the physical actuation system and overall device interaction using the updated runtime logging framework. Because no calibrated force sensor or phase-current measurement was available, the tests did not measure physical output force directly. Instead, logged position, proxy position, rendered force command, host torque command, and device-side applied torque were used as quantitative indicators of contact behaviour, bounded motion, command consistency, and practical interaction quality.

4.4.1 Repeated Contact Test

A repeated-contact test evaluated the behaviour of the full closed-loop system during sustained interaction with a virtual surface.

The end-effector was brought into contact with a virtual object while actuator feedback was enabled. Logged data were used to assess whether penetration remained bounded and whether the commanded force–penetration relationship remained consistent with the implemented virtual coupling model. The reported metrics were mean, percentile, and maximum penetration depth, together with the fitted force–penetration stiffness and coefficient of determination. Qualitative observations of oscillation, chatter, and usability were used only to contextualise the logged results.

4.4.2 Constrained Motion Hardware Test

Hardware constrained-motion behaviour was evaluated during physical operation of the device. The end-effector was brought into contact with a planar virtual surface and moved along it while maintaining contact. Quantitative metrics were computed over a sustained-contact window, excluding entry and release transients. The evaluation assessed whether the logged force command was predominantly normal to the surface, whether normal displacement remained bounded, and whether tangential motion could be produced without large oscillatory excursions. The reported metrics were mean and maximum penetration, normal-to-tangential force ratio, sustained-contact force magnitude, and force coefficient of variation.

5 Results

This section reports the measured outcomes of the Section 4 tests. It presents communication timing, simulation behaviour, safety outcomes, and hardware interaction observations. Interpretation of these findings and their design implications is deferred to Section 6.

5.1 Communication Performance

Communication performance was evaluated using standalone serial tests and runtime logging of the integrated haptic application.

5.1.1 Packet Rate and Integrity Results

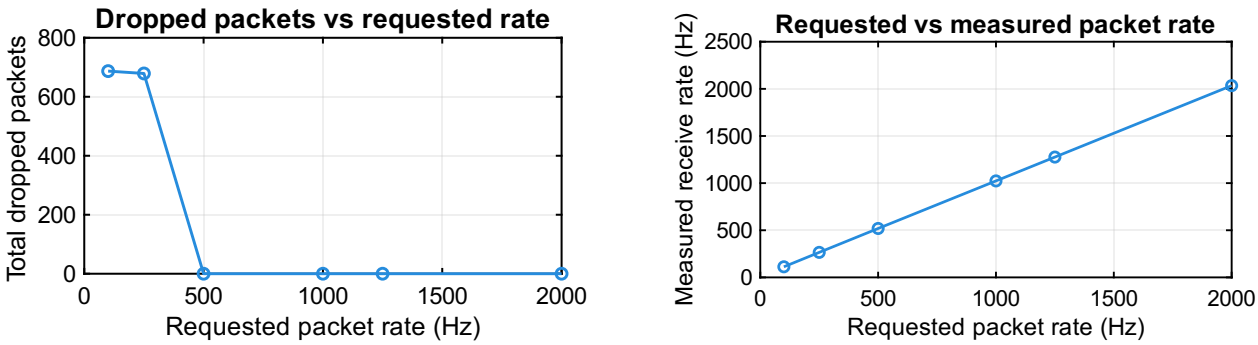
Packet rate testing showed that communication performance remained consistent over a wide range of requested transmission frequencies, although the behaviour was not uniform across all tested rates.

At lower requested rates of 100 Hz and 250 Hz, packet loss was observed, with drop fractions of approximately 20.3% and 8.5% respectively. In contrast, for requested rates of 500 Hz and above, no dropped packets were observed in the recorded data, as shown in Fig. 3(a), where packet loss occurs only at the lowest transmission rates. Out-of-order packets were extremely rare across all higher-rate tests, with only a single out-of-order event recorded in each case.

The measured receive rate tracked the requested transmission rate closely across the full test range. As shown in Fig. 3(b), the relationship between requested and measured rate was approximately linear, indicating that the communication system maintained throughput proportional to the commanded transmission frequency.

Inter-arrival jitter, measured as the standard deviation of packet inter-arrival time, was present in all tests, with standard deviation decreasing as the requested rate increased, as shown in Fig. 4. However, percentile analysis revealed that occasional large timing excursions persisted even at higher rates, indicating intermittent latency spikes. While average jitter decreased with increasing rate, worst-case timing excursions remained present.

Overall, the serial link supported nominal packet throughput at or above 1 kHz with negligible packet



(a) Dropped packets

(b) Rate tracking

Fig. 3. Communication performance across transmission rates. Packet loss occurred only at low frequencies, while measured receive rate tracked the requested rate approximately linearly.

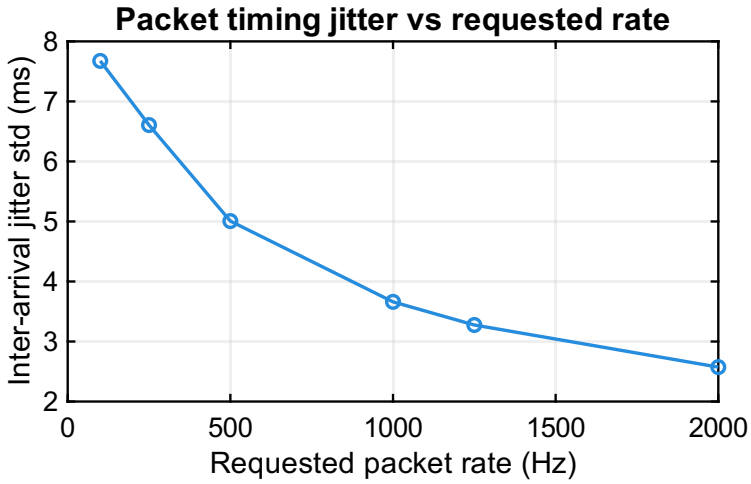


Fig. 4. Packet inter-arrival jitter decreases with requested rate, although occasional large timing excursions remain present.

loss in the tested configuration. This result describes packet generation and throughput, rather than low-latency delivery or 1 ms closed-loop state freshness.

5.1.2 Round-Trip Latency Results

Round-trip latency results were highly consistent across all tested ping intervals. For nominal transmission intervals of 1 ms, 2 ms, and 10 ms, the success rate was 100% in all cases, with no timeouts observed over 1000 samples per test. The mean measured round-trip latency remained approximately constant across all three test conditions, with values of 15.60 ms, 15.64 ms, and 15.67 ms respectively. Similarly, the 95th and 99th percentile RTT values were tightly grouped around 16.2–16.4 ms, indicating consistent and predictable timing behaviour across repeated measurements. This behaviour is further illustrated in Fig. 5, where the cumulative distributions for all three tests overlap closely and rise sharply at approximately 15–16 ms, indicating that RTT is largely independent of the selected ping interval.

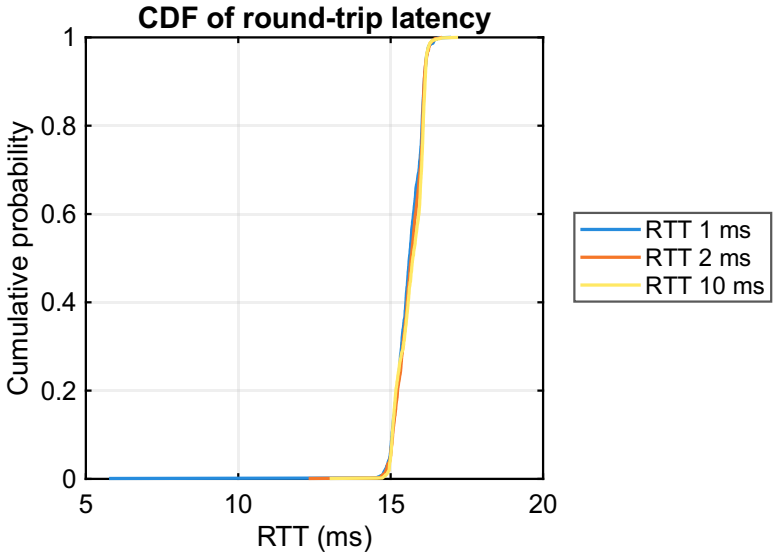


Fig. 5. Cumulative distributions of RTT for 1 ms, 2 ms, and 10 ms ping intervals. The overlapping curves indicate a largely fixed 15–16 ms delay component.

The standard deviation of RTT remained below 0.52 ms in all cases, suggesting relatively low variability in round-trip timing. Minimum and maximum observed RTT values showed occasional larger deviations from the typical latency range, particularly in the 1 ms test, but these did not significantly affect the overall distribution.

These results indicate that RTT was dominated by a largely fixed delay component and changed little with ping interval. The measured delay remained predictable due to low variability and zero observed timeouts.

5.1.3 End-to-End Host-Side Timing Results

End-to-end timing behaviour of the integrated haptic pipeline was evaluated using runtime logging of matched state-command cycles and raw parsed state packets.

Analysis of `device_timing.csv` showed that all recorded control cycles were successfully matched, with a matched fraction of 1.000, indicating consistent correspondence between received state packets and transmitted torque commands.

The host-side end-to-end latency, defined as the time between state-packet parsing and completion of torque command transmission, was found to be low. The mean latency was 0.299 ms, with a median of 0.104 ms. The 95th percentile latency was 0.546 ms, and the 99th percentile was 2.04 ms, with a maximum observed latency of 3.35 ms. As shown in Fig. 6, the distribution was concentrated at low values, with prominent peaks near 0 ms and around 0.5 ms, and only a limited number of

higher-latency events.

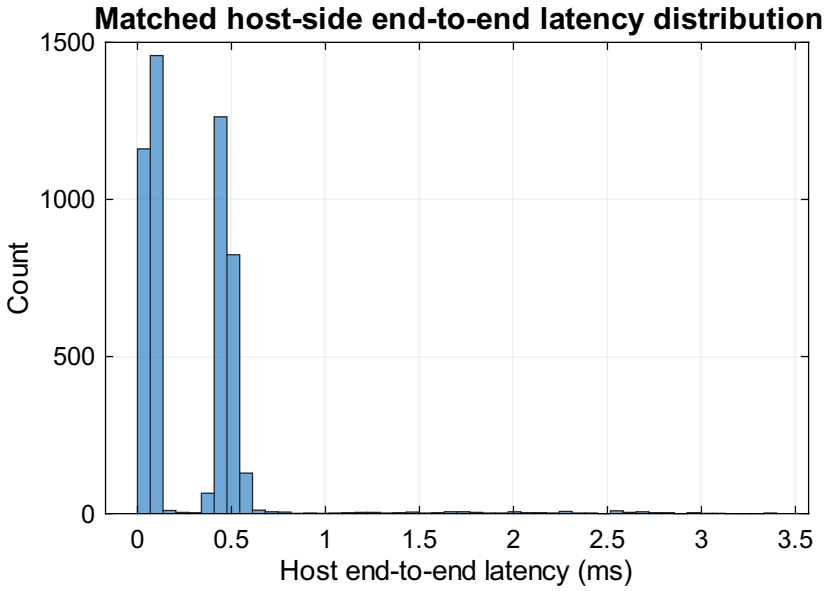


Fig. 6. Matched host-side latency from state-packet parsing to torque-command transmission, concentrated at low values with few higher-latency outliers.

Breakdown of intermediate pipeline delays showed that the majority of this latency was associated with serial transmission. The mean transmission duration was 0.249 ms, while the delay associated with publishing tool state and processing the haptic pipeline remained small, with mean values of 0.024 ms and 0.0044 ms respectively. This indicates that host-side computation and message-passing overhead were negligible relative to communication time.

Sequence analysis showed that command sequence progression remained consistent, with a mean command sequence gap of 1.14 and a maximum of 6, indicating that command updates were generally applied sequentially with minimal skipping. In contrast, the matched state sequence gap was significantly larger, with a mean of 33.45. This reflects the fact that the control loop operated on a subset of the available incoming state packets rather than processing every parsed packet. Here, sequence gap denotes the difference between consecutive sequence identifiers, such that a value of 1 corresponds to consecutive packets or commands and larger values indicate skipped intermediate sequence numbers.

A total of 54 161 valid state packets were parsed during the test, compared to 5 048 matched control cycles, confirming that the control loop operated on a subset of the incoming packet stream rather than consuming every parsed packet. The estimated parsed packet rate, computed from the mean host-side inter-arrival time, was approximately 1006 Hz, which is close to the expected 1 kHz transmission rate.

Analysis of host-side parsed-packet inter-arrival timing revealed significant variability. The mean inter-arrival time was 0.994 ms, but the median was only 0.0016 ms, reflecting that many packets were processed in immediate succession within bursts. The large difference between mean and median inter-arrival time indicates that packets were frequently processed in rapid bursts following periods of delay, rather than arriving at uniform intervals. This behaviour is illustrated in Fig. 7, where a dominant peak near 0 ms is accompanied by a much smaller secondary concentration around 14–16 ms. The 95th and 99th percentile inter-arrival times were 14.5 ms and 15.8 ms respectively, confirming the presence of intermittent host-side stalls.

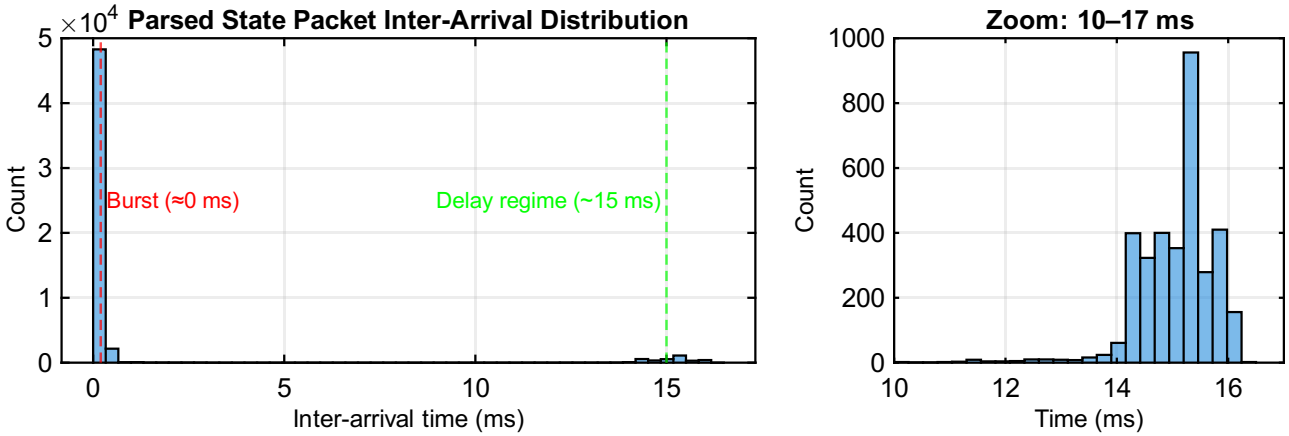


Fig. 7. Host-side parsed-packet inter-arrival distribution. The secondary peak corresponds to the 95th–99th percentile range of 14.5–15.8 ms.

Despite this non-uniform host-side processing pattern, analysis of MCU-side timestamps showed that the embedded system generated state packets at a consistent rate. The effective MCU period per state was approximately 1.00 ms, corresponding to a nominal packet generation rate of 1000 Hz, with very low variability.

Taken together, the embedded controller generated packets at a regular rate, while host-side packet timing was non-uniform. Host-side processing latency nevertheless remained low. These results separate local packet generation and host computation from the effective age of the state used for haptic feedback.

5.2 Simulation Validation Results

Simulation validation was performed as described in Section 4.2, with actuator output disabled. These tests evaluated the haptic rendering model independently of physical actuator dynamics.

5.2.1 Virtual Wall Contact Response

The virtual wall response was evaluated using a single-depth hold test. During contact, the rendered force magnitude had a mean value of 11.46 N and a standard deviation of 0.30 N, corresponding to a coefficient of variation of 0.026.

As shown in Fig. 8, force was generated only when the end-effector penetrated the virtual wall. When the end-effector was outside the wall, the output force remained approximately zero.

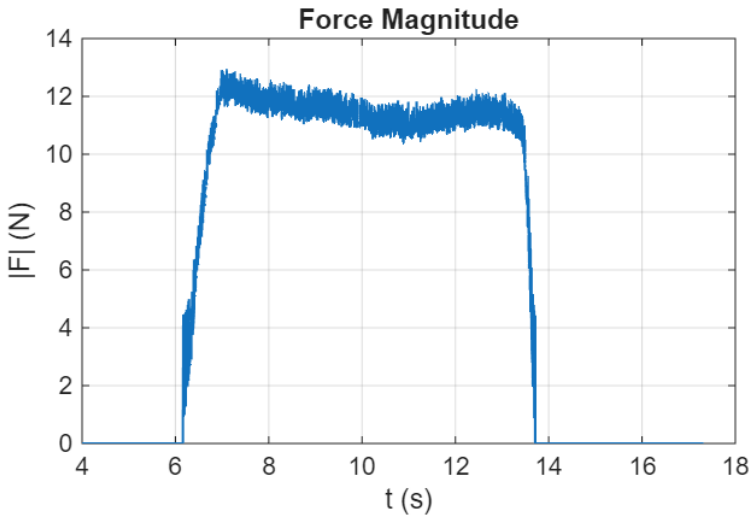


Fig. 8. Single-depth virtual wall contact test. The rendered $|\mathbf{F}|$ remains steady during sustained contact, with mean 11.46 N, standard deviation 0.30 N, and coefficient of variation 0.026.

5.2.2 Force–Penetration Characterisation

A multi-depth hold test was used to quantify the relationship between virtual penetration depth and logged rendered-force magnitude. A linear fit to the force–penetration data produced an effective stiffness of 489.57 N m^{-1} with $R^2 = 0.9973$, as shown in Fig. 9.

This closely matches the configured virtual coupling stiffness of 500 N m^{-1} over the tested range.

5.2.3 Free-Space and Saturation Behaviour

A no-contact test verified that the haptic engine produced no significant force output in free space.

A deep-penetration test was also performed to verify host-side force limiting. Under large penetration, the commanded force saturated at approximately 31 N, as shown in Fig. 10.

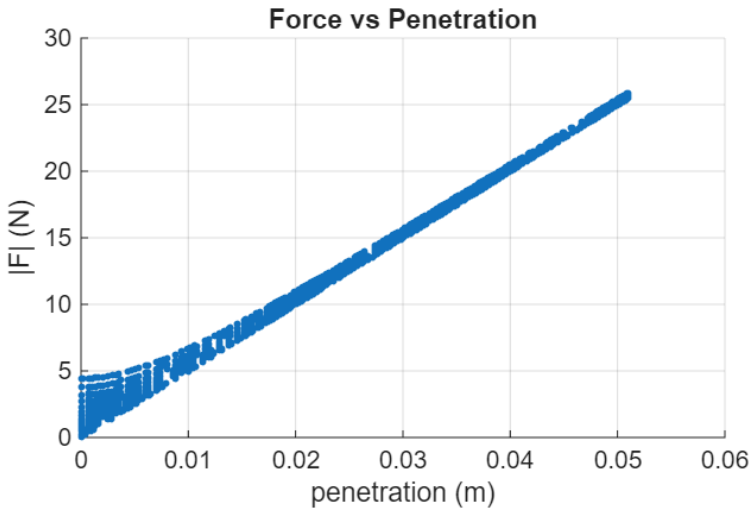


Fig. 9. Force–penetration relationship from the multi-depth hold test, with fitted stiffness 489.57 N m^{-1} and $R^2 = 0.9973$.

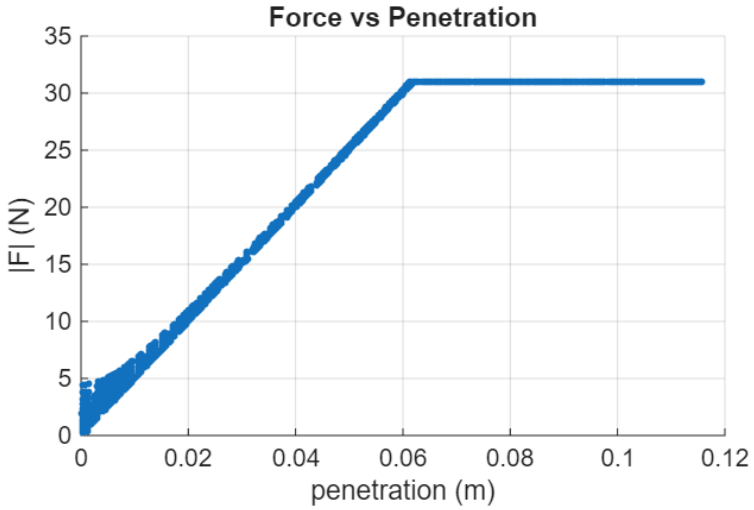


Fig. 10. Deep-penetration simulation test showing force saturation at the host-side limit of approximately 31 N.

5.3 Safety Validation Results

Safety validation evaluated whether torque commands remained bounded during high-force contact, rapid command transitions, and communication loss.

5.3.1 Torque Saturation

Torque saturation was evaluated using the deep-penetration test. Host-side saturation occurred in 31.10% of samples, with the maximum raw torque command reaching 9.35 N m before being clamped to 7.66 N m. On the device side, the maximum applied torque was limited to 6.50 N m, matching the configured firmware torque cap.

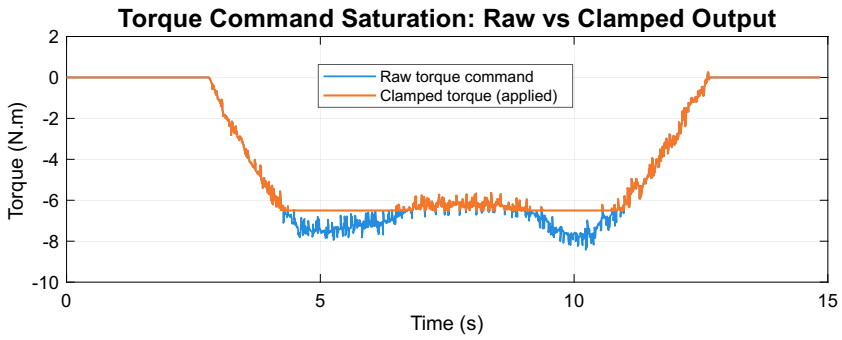


Fig. 11. Torque saturation during deep-penetration testing: raw host commands are clamped before transmission and applied torque remains bounded by the 6.50 N m firmware cap.

5.3.2 Rate Limiting

The embedded torque slew-rate limiter was evaluated from the applied torque logs. The measured 95th percentile slew rate was 55.000 N m s^{-1}, the 99th percentile was 55.005 N m s^{-1}, and the maximum measured slew rate was 55.010 N m s^{-1}. These values closely match the configured firmware limit of 55 N m s^{-1}.

The slew-rate distribution is shown in Fig. 12. The dominant peak at 0 N m s^{-1} corresponds to steady-state operation, while the sharp upper bound occurs at the configured 55 N m s^{-1} limit.

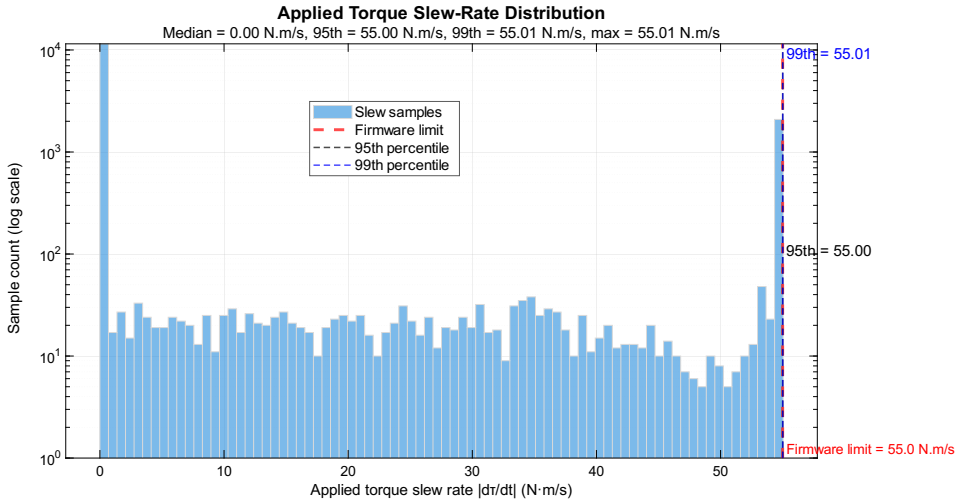


Fig. 12. Applied torque slew-rate histogram showing a sharp upper bound at the firmware limit of 55 N m s^{-1}.

5.3.3 Watchdog Timeout Behaviour

The watchdog was evaluated by deliberately interrupting host torque-command transmission. A total of 259 watchdog activation events were recorded, corresponding to 6.81% of device samples.

The total watchdog-active time was 0.529 s, with a mean activation duration of 0.0020 s and a maximum duration of 0.311 s.

As shown in Fig. 13, applied torque was removed during watchdog activation.

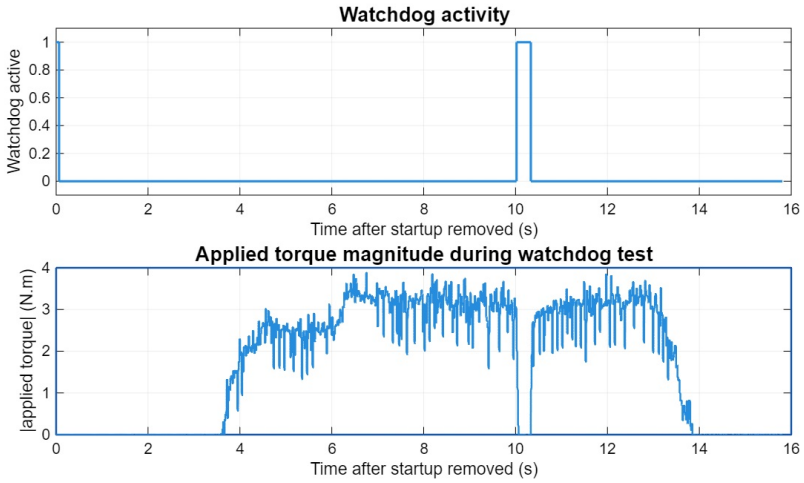


Fig. 13. Watchdog validation during interrupted host command transmission. Applied torque is removed when the watchdog becomes active.

5.3.4 Command Consistency

Sequence-matched host and device logs showed a mean applied torque magnitude error of 0.134 N m between host commands and device-side applied torque. For typical commanded torques of approximately 2–4 N m, this corresponds to roughly 3–7% error. Larger differences occurred during transient command changes, especially when the slew-rate limiter and safety clamps were active.

5.4 Hardware Validation Results

Hardware validation was performed with actuator feedback enabled. Because direct force sensing and current measurement were not available, these results quantify interaction behaviour using logged position, force-command, and torque-command data rather than calibrated physical force output. Final force-feedback testing was limited to single-axis actuation due to actuator failure, although both joints remained available for position sensing.

5.4.1 Repeated Contact With Virtual Wall

During repeated motor-enabled contact with the virtual wall, penetration remained bounded. The mean penetration depth was 0.0342 m, the 95th percentile penetration depth was 0.0498 m, and the maximum penetration depth was 0.0517 m.

The force–penetration relationship during this test remained approximately linear, with an effective stiffness of 490.99 N m^{-1} and $R^2 = 0.9979$. This indicates that the rendered force command remained consistent with the virtual coupling model during closed-loop interaction. The fitted stiffness of 490.99 N m^{-1} agrees closely with the simulation-only result of 489.57 N m^{-1} in Section 5.2.2. This comparison is based on logged command data rather than direct force measurement.

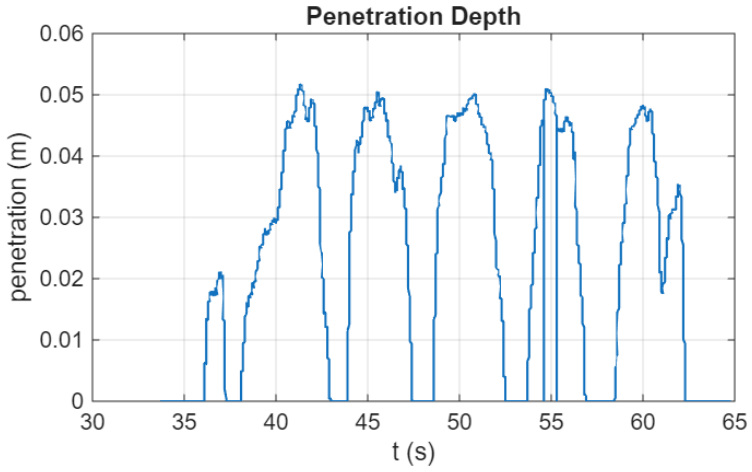


Fig. 14. Motor-enabled repeated contact test. Maximum penetration was approximately 0.052 m; the force–penetration fit gave 490.99 N m^{-1} with $R^2 = 0.9979$.

5.4.2 Constrained Motion Along a Surface

Constrained motion was evaluated by bringing the end-effector into contact with the virtual wall and moving along the surface. Metrics were computed over the sustained-contact window from 20.50 s to 23.20 s, excluding entry and release transients. During this window, the mean penetration depth, computed from device–proxy separation, was 0.0492 m and the maximum penetration depth was 0.0512 m.

Force decomposition showed that the rendered force was predominantly normal to the surface, with a median $|F_n|/|F_t|$ ratio of 17.72 and a 95th percentile ratio of 266.03. The sustained-contact force magnitude had a mean value of 24.68 N and a standard deviation of 0.72 N, corresponding to a coefficient of variation of 0.029.

The device and proxy trajectories are shown in Fig. 15. The proxy remains aligned with the virtual wall, while the device trajectory shows bounded displacement normal to the wall and motion along the tangential direction.

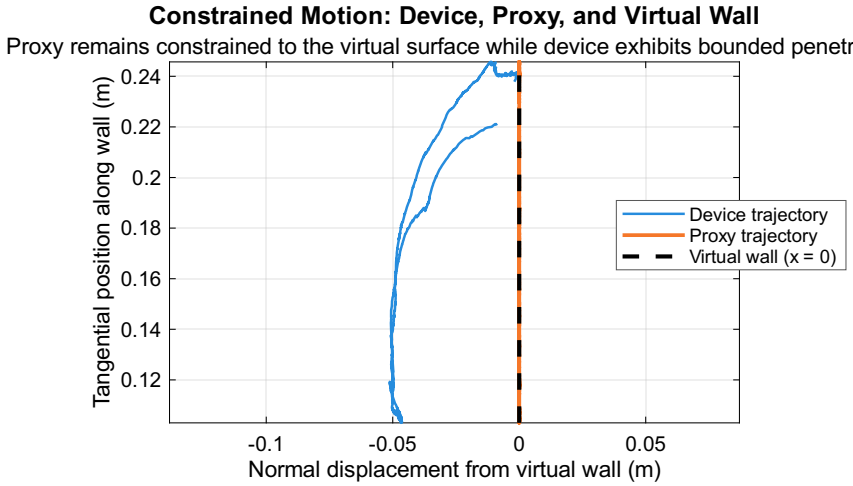


Fig. 15. Device and proxy trajectories during the analysed constrained-motion window from 20.50 s to 23.20 s. The proxy remains constrained to the wall while device normal displacement stays bounded.

Across the motor-enabled hardware tests, penetration remained bounded and logged force-command behaviour remained approximately consistent with the implemented virtual coupling model.

6 Discussion

This section interprets the experimental results in terms of the main constraints on haptic performance: communication latency, delayed contact behaviour, hardware limitations, and the resulting design trade-offs.

6.1 Communication and Latency Constraints

The central finding from the communication evaluation is that transport delay, measured at 15.60–15.67 ms mean round-trip with 95th and 99th percentiles around 16.2–16.4 ms, is the dominant limitation on closed-loop performance. In contrast, host-side processing latency remained low (mean 0.299 ms, 99th percentile 2.04 ms), confirming that computation is not the bottleneck for the nominal 1 kHz local haptic/device loops.

This delay directly bounds achievable haptic stiffness. As delay increases, phase lag reduces the stability margin and can inject artificial energy into the sampled-data loop [16], so lower stiffness and

additional damping are required. Consequently, the realised system renders compliant rather than rigid contact.

The ST-Link virtual COM interface was convenient for development, but the observed burst-like delivery pattern and low-rate packet loss suggest USB CDC buffering and host scheduling overhead that is large relative to the control timestep. Although the present work does not isolate every stage in the communication path, the evidence shows that communication architecture must be treated as a first-order design constraint alongside control and mechanical design.

The system should therefore be interpreted as a high-rate local haptic renderer operating over delayed sampled state, rather than as a true 1 kHz end-to-end haptic loop. Snapshot channels reduce backlog-induced latency by ensuring that the haptic and device loops consume the newest available state, but they cannot remove transport delay, increase the physics-simulation update rate, or guarantee that the rendered contact state is physically fresh at 1 ms intervals.

6.2 Local Contact State Limitation

A further limitation of the realised architecture is that the nominal 1 kHz haptic loop did not evolve a complete local contact state independently of the slower simulation and communication layers. It consumed latest-available snapshots of tool and world state, then recomputed proxy/contact information at each cycle. The renderer could therefore execute at high frequency, but the physical meaning of each update was still bounded by snapshot freshness and communication delay. The limitation was not the execution rate of the haptic computation, but that the rendered state was not itself evolved and validated at 1 kHz across the full interaction loop.

A more complete architecture would maintain persistent proxy and local contact state inside the high-rate loop. Rather than re-projecting the proxy from scratch, the proxy would evolve continuously subject to contact constraints, preserving contact history, tangential motion, and release behaviour. This is closer to constraint-based proxy or god-object approaches [3], [12], [15]. The simpler projection-based approach used here was sufficient to demonstrate bounded contact, but it limited contact smoothness and weakens any claim to true 1 kHz end-to-end haptic interaction.

This clarifies the relationship between the haptic renderer and the physics engine. The physics engine provided the authoritative global scene state at a lower rate, while the haptic loop consumed snapshots of that state. A stronger version of this multi-rate design would allow the haptic loop to

maintain a reduced local model of contacted objects at high frequency, while slower application, rendering, or physics subsystems provide broader scene updates and maintain global consistency, following the motivation of multilevel haptic rendering for complex environments [3].

6.3 Observed Contact Behaviour Under Delay

Although communication timing was imperfect, the tested interactions remained bounded and controllable because snapshot channels used the newest available state rather than queued historical samples, thread separation avoided cross-subsystem blocking, and damping countered delayed feedback effects. Embedded torque rate limiting further prevented abrupt torque transitions that can excite oscillations. Watchdog activation data also show that communication interruptions were handled safely: 259 activation events were recorded, with a mean duration of 2 ms and torque removed during activation (Section 5).

These mechanisms address the passivity concerns identified by Colgate and Schenkel [16] in a practical rather than formal way. The results provide evidence of bounded behaviour in the tested cases, not a passivity proof. Contact therefore remained controllable without divergent motion or sustained oscillation, but the compliant feel was still a direct consequence of delayed feedback; rigid contact cannot be achieved under the measured latency conditions by force-model tuning alone.

6.4 Hardware Limitations

The hardware results must be interpreted within clear prototype constraints. The most important was the absence of current sensing. The initial design targeted STM32 B-G431-ESC1 motor drivers, but integration with the selected magnetic encoders proved unreliable within the project timescale. SimpleFOC Mini modules provided a more robust integration path, but left torque control effectively open-loop with respect to current. As a result, commanded torque cannot be interpreted as calibrated physical torque, and the hardware evaluation should be read primarily as evidence of logged command consistency and qualitative interaction capability. This limitation does not affect the communication, timing, and bounded-contact results, which do not depend on absolute force calibration.

A second limitation was the GT2 belt reduction used to increase base-joint torque. The belt improved available torque but slipped under high load, even after adding an adjustable idler pulley,

probably because 3D-printed pulleys did not provide manufactured tooth-profile precision. Actuator torque limits would still bound force output even without slip, directly constraining renderable stiffness.

A further limitation arose from the failure of one actuator during development. Final hardware validation therefore used single-joint actuation with two-joint sensing, so conclusions about full 2-DOF force rendering remain provisional. Overall, actuator capability and transmission design impose significant constraints alongside communication latency in low-cost haptic systems.

6.5 Design Trade-Offs

The system therefore reflects a trade-off between bounded contact behaviour, responsiveness, and force fidelity. Damping and torque rate limiting avoided abrupt or divergent responses, but reduced perceived stiffness. The measured limiter of approximately 55 N m s^{-1} implies an approximate 0.13 s rise time to reach the 7 N m torque range, so hard contacts feel gradual rather than instantaneous.

An additional perceptual limitation was observed at contact release: after sustained penetration into a virtual wall, the transition back to free motion could feel slightly jerky. This may be associated with abrupt proxy-state changes, lag in the filtered proxy velocity state, or delayed torque commands. The implemented approach therefore prioritised controllability over maximum stiffness, showing that high-fidelity force rendering cannot be improved by optimising one subsystem in isolation.

6.6 Project Management and Development Process

The project plan provided a useful structure, but the final Gantt chart in Appendix B reflects an adaptive process shaped by integration risk. The main management decision was to prioritise a working system-level prototype over ideal subsystem performance: driver integration difficulties led to SimpleFOC Mini modules, and later actuator failure shifted final validation from full 2-DOF force feedback to single-axis actuation with two-axis sensing. Risk management therefore focused on preserving the communication, simulation, safety, and bounded-contact evaluation objectives through staged testing, torque limiting, slew-rate limiting, and watchdog safety.

7 Conclusions and Future Work

This section summarises the extent to which the project objectives were achieved and identifies the most important directions for future development.

7.1 Conclusions

This project presented the design, implementation, and evaluation of a low-cost, modular haptic rendering system integrating a two-degree-of-freedom input device with a custom real-time simulation and visualisation framework. The system successfully achieved its primary objective of enabling bidirectional interaction between a user-operated device and a virtual environment, with real-time computation of interaction forces and feedback through actuator torque control. The architecture was designed to prioritise modularity and extensibility, allowing clear separation between hardware, communication, simulation, and rendering subsystems.

Experimental evaluation showed that selected local subsystems could operate at nominal 1 kHz rates, including embedded packet generation and host-side haptic/device-loop execution, with low host-side processing latency. However, the complete bidirectional haptic interaction should not be claimed as a true 1 kHz end-to-end loop, because force computation operated on latest-available snapshots affected by communication delay and asynchronous simulation timing. The dominant limitation was the 15.60–15.67 ms round-trip communication delay, which bounded achievable virtual stiffness. Within this constraint, bounded and controllable contact interaction was observed during the tested interactions through a combination of architectural and control strategies, including snapshot-based communication and damping-based stabilisation. The resulting haptic feedback was usable and controllable, but not rigid or quantitatively force-calibrated, due to finite actuator torque, the absence of current sensing, belt slip under high load, and the limitation to single-axis actuation during final validation.

The project outcomes address the objectives defined in Section 1. A sub-£200 haptic interface was constructed with joint sensing and motor command capability, although final actuation was limited to one joint after actuator failure. The system demonstrated nominal 1 kHz packet streaming while separately quantifying packet loss, out-of-order events, burst-like delivery, RTT, and host-side processing latency. Logged simulated rendered-force data gave an effective stiffness of 489.57 N m^{-1}, within 5% of the configured 500 N m^{-1} value, with $R^2 = 0.9973$. Saturation, torque limiting, slew-

rate limiting, watchdog behaviour, motor-enabled contact, and constrained-motion tests were also validated, while documenting the communication, actuation, sensing, and mechanical limitations that affect generalisation.

Overall, the project demonstrates that bounded and usable haptic interaction can be achieved using low-cost hardware and a modular software architecture, even in the presence of significant communication latency. The main insight is that high-rate local control and an accurate command-side force model are not sufficient on their own: communication delay, actuator capability, transmission design, and the absence of a persistent high-rate local contact-state model jointly determine achievable haptic fidelity.

7.2 Future Work

A key priority is reducing communication latency. The current ST-Link virtual COM pathway appears to introduce buffering and scheduling delays that dominate the closed-loop response, so future work should compare lower-latency alternatives such as direct UART via a dedicated USB–UART bridge, native USB bulk transfer, CAN bus, or Ethernet-based communication.

Future work should extend the existing multi-rate architecture so that the haptic loop maintains a reduced local model of currently contacted objects at 1 kHz, while slower physics, rendering, or application layers remain responsible for broader scene evolution and consistency. The high-rate loop would continuously evolve the proxy, contact manifold, and local object interaction state, reducing dependence on low-rate world snapshots and improving contact continuity, following the motivation of multilevel haptic rendering for complex environments [3].

A related extension would be to move the haptic rendering loop, or at least the local contact/proxy evolution, onto the embedded controller. Communication with the host would then focus on object interaction parameters, local contact primitives, or periodic scene-state updates, rather than every force update depending on a delayed host–device exchange. This would lower effective actuator-side latency, with the trade-off that the embedded model would need to be simplified using local planes, SDF patches, virtual fixtures, or reduced contact geometry.

On the control side, incorporating task-space dynamics using an operational-space formulation [13] would allow the manipulator's inertia and dynamics to be compensated during interaction, improving force-rendering transparency.

Hardware improvements are also important. Current sensing, together with external force measurement, would enable more accurate torque control, quantitative physical force validation, and more advanced impedance or operational-space control. Transmission design could be improved by replacing the slipping GT2 belt stage with a lower-backlash alternative such as a capstan drive, while higher-torque actuators and restored two-degree-of-freedom actuation would improve interaction realism.

Further improvements to the haptic rendering model could address the force-direction inconsistencies observed during large penetration. Constraint-based contact solvers could enforce continuous proxy motion across timesteps, reducing surface-normal discontinuities caused by re-projecting the proxy from scratch. Contact entry and exit transitions should also be smoothed by blending proxy velocity state across release and discarding stale torque commands before they affect the user.

From a software and evaluation perspective, integration with platforms such as NVIDIA Isaac Sim or MATLAB/Simulink would enable more realistic robotic scenarios, while quantitative force measurement, user studies, and task-based metrics would better assess effectiveness as a training tool.

Together, these developments would enable the system to evolve from a low-cost prototype into a more capable and general-purpose haptic interface platform for robotic interaction and training applications.

References

[1] G. S. Giri, Y. Maddahi, and K. Zareinia, "An Application-Based Review of Haptics Technology," *Robotics*, vol. 10, no. 1, p. 29, Feb. 5, 2021, ISSN: 2218-6581. DOI: 10.3390/robotics10010029 Accessed: Apr. 16, 2026. [Online]. Available: https://www.mdpi.com/2218-6581/10/1/29 (cited on pp. 1, 5, 8).

[2] M. Bergholz, M. Ferle, and B. M. Weber, "The benefits of haptic feedback in robot assisted surgery and their moderators: A meta-analysis," *Scientific Reports*, vol. 13, no. 1, p. 19 215, Nov. 6, 2023, ISSN: 2045-2322. DOI: 10.1038/s41598-023-46641-8 Accessed: Apr. 16, 2026. [Online]. Available: https://www.nature.com/articles/s41598-023-46641-8 (cited on p. 1).

[3] D. C. Ruspini, K. Kolarov, and O. Khatib, "The haptic display of complex graphical environments," in *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques - SIGGRAPH '97*, Not Known: ACM Press, 1997, pp. 345–352, ISBN: 978-0-89791-896-1. DOI: 10.1145/258734.258878 Accessed: Apr. 16, 2026. [Online]. Available: http://portal.acm.org/citation.cfm?doid=258734.258878 (cited on pp. 1, 5, 6, 13, 30, 31, 34).

[4] R. Adams and B. Hannaford, "Stable haptic interaction with virtual environments," *IEEE Transactions on Robotics and Automation*, vol. 15, no. 3, pp. 465–474, Jun. 1999, ISSN: 1042296X. DOI: 10.1109/70.768179 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/768179/ (cited on pp. 2, 6, 7, 13).

[5] M. O. Martinez et al., "3-D printed haptic devices for educational applications," in *2016 IEEE Haptics Symposium (HAPTICS)*, Apr. 2016, pp. 126–133. DOI: 10.1109/HAPTICS.2016.7463166 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/7463166 (cited on pp. 2, 4, 8).

[6] M. O. Martinez et al., "Open source, modular, customizable, 3-D printed kinesthetic haptic devices," in *2017 IEEE World Haptics Conference (WHC)*, Jun. 2017, pp. 142–147. DOI: 10.1109/WHC.2017.7989891 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7989891 (cited on pp. 2, 4, 8).

[7] A. Lavatelli, M. Bordegoni, and F. Ferrise, "Design of an open-source low cost 2DOF haptic device," Sep. 2014. DOI: 10.1109/MESA.2014.6935574 (cited on pp. 2, 4, 8).

[8] “Geomagic touch haptic device,” Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-haptic-device (cited on p. 5).

[9] “Geomagic touch X haptic device,” Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-x-haptic-device (cited on p. 5).

[10] “Sales inquiries,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.forcedimension.com/contact/sales (cited on p. 5).

[11] “Inverse3 X,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.haply.co/inverse3x (cited on p. 5).

[12] C. Zilles and J. Salisbury, “A constraint-based god-object method for haptic display,” in *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*, vol. 3, Pittsburgh, PA, USA: IEEE Comput. Soc. Press, 1995, pp. 146–151, ISBN: 978-0-8186-7108-1. DOI: 10.1109/IROS.1995.525876 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/525876/ (cited on pp. 5, 6, 13, 30).

[13] O. Khatib, “A unified approach for motion and force control of robot manipulators: The operational space formulation,” *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, Feb. 1987, ISSN: 2374-8710. DOI: 10.1109/JRA.1987.1087068 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/1087068 (cited on pp. 6, 34).

[14] C. Ericson, *Real-Time Collision Detection*, 0th ed. CRC Press, Dec. 22, 2004, ISBN: 978-0-08-047414-4. DOI: 10.1201/b14581 Accessed: Apr. 16, 2026. [Online]. Available: https://www.taylorfrancis.com/books/9780080474144 (cited on pp. 6, 13).

[15] K. Salisbury and C. Tarr, “Haptic Rendering of Surfaces Defined by Implicit Functions,” in *Dynamic Systems and Control*, Dallas, Texas, USA: American Society of Mechanical Engineers, Nov. 16, 1997, pp. 61–67, ISBN: 978-0-7918-1824-4. DOI: 10.1115/IMECE1997-0378 Accessed: Apr. 16, 2026. [Online]. Available: https://asmedigitalcollection.asme.org/IMECE/proceedings/IMECE97/18244/61/1138197 (cited on pp. 6, 13, 30).

[16] J. E. Colgate and G. G. Schenkel, “Passivity of a class of sampled-data systems: Application to haptic interfaces,” *Journal of Robotic Systems*, vol. 14, no. 1, pp. 37–47, 1997, ISSN: 1097-4563. DOI: 10.1002/(SICI)1097-4563(199701)14:1<37::AID-ROB4>3.0.CO;2-V Accessed: Apr. 18, 2026. [Online]. Available: https://onlinelibrary.wiley.com/doi/

abs/10.1002/%28SICI%291097-4563%28199701%2914%3A1%3C37%3A%3AAID-ROB4%3E3.0.CO%3B2-V (cited on pp. 7, 29, 31).

[17] L. Ramadoss and J. Y. Hung, "A study on universal serial bus latency in a real-time control system," in *2008 34th Annual Conference of IEEE Industrial Electronics*, Nov. 2008, pp. 67–72. DOI: 10.1109/IECON.2008.4757930 Accessed: May 4, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/4757930 (cited on p. 7).

[18] T-Motor, *GB2208 gimbal type brushless motor*, 2026. [Online]. Available: https://store.tmotor.com/product/gb2208-gimbal-type.html (cited on p. 10).

[19] ams AG, *AS5048A/AS5048B magnetic rotary encoder (14-bit angular position sensor)*, v1-11, manual, ams AG, Jan. 2018. Accessed: Apr. 16, 2026. [Online]. Available: https://ams-osram.com/products/sensors/position-sensors/ams-as5048a-high-resolution-position-sensor (cited on p. 10).

[20] ams AG, *AS5600 12-bit programmable contactless potentiometer*, v1-06, manual, ams AG, Jun. 2018. Accessed: Apr. 16, 2026. [Online]. Available: https://ams-osram.com/products/sensor-solutions/position-sensors/ams-as5600-position-sensor (cited on p. 10).

[21] "NUCLEO-G431RB | Product - STMicroelectronics," Accessed: Apr. 16, 2026. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-g431rb.html (cited on p. 11).

[22] A. Skuric et al., "SimpleFOC: A field oriented control (FOC) library for controlling brushless direct current (BLDC) and stepper motors," *Journal of Open Source Software*, vol. 7, no. 74, p. 4232, 2022. DOI: 10.21105/joss.04232 [Online]. Available: https://doi.org/10.21105/joss.04232 (cited on p. 11).

[23] P. Vas, *Sensorless Vector and Direct Torque Control*. Oxford University Press, May 14, 1998, ISBN: 978-0-19-856465-2. DOI: 10.1093/oso/9780198564652.001.0001 Accessed: Apr. 16, 2026. [Online]. Available: https://doi.org/10.1093/oso/9780198564652.001.0001 (cited on p. 11).

Appendices

A Preliminary Project Proposal

This appendix records the initial project outline submitted at the start of the work, providing context for how the project scope evolved during development.

Design and Development of a Low-Cost 2-DOF Haptic Training Interface for Robotic Manipulator Simulation

Tyler Ward

October 2025

1 Background and Motivation

Robotic manipulators are increasingly used in manufacturing, surgical, and remote handling applications. However, operator training often relies on visual feedback alone, limiting spatial understanding and increasing the risk of errors when transitioning to physical systems.

Haptic feedback provides tactile and force based sensations that improve the realism and safety of training by allowing users to feel resistance and contact forces. Commercial haptic devices are often expensive and closed source, which makes them unsuitable for flexible training or academic development.

This project aims to create a low cost modular 2 DOF haptic input device that can send user motion commands to a simulated robotic manipulator and receive realistic force feedback. The system will be designed to support future integration with real robotic hardware, providing an accessible and scalable training platform.

2 Aim

To design, prototype and evaluate a compact low cost two degree of freedom haptic interface that can communicate in real time with a robotic manipulator simulation and provide accurate position sensing and responsive force feedback to the user.

3 Objectives

1. **Literature and Technology Review**
   A review will be carried out to identify existing haptic devices and relevant control methods. This will guide the selection of motors, encoders, and actuation strategies suitable for a low cost modular interface.
2. **Physics Simulation Development**
   A physics based simulation environment will be designed to model manipulator dynamics, haptic interactions, and constraint behavior.
3. **Hardware Design**

1

Fig. 16. Original project outline as submitted at the start of the project

B Gantt Chart

This appendix shows the final project schedule and the main development phases completed during the project.

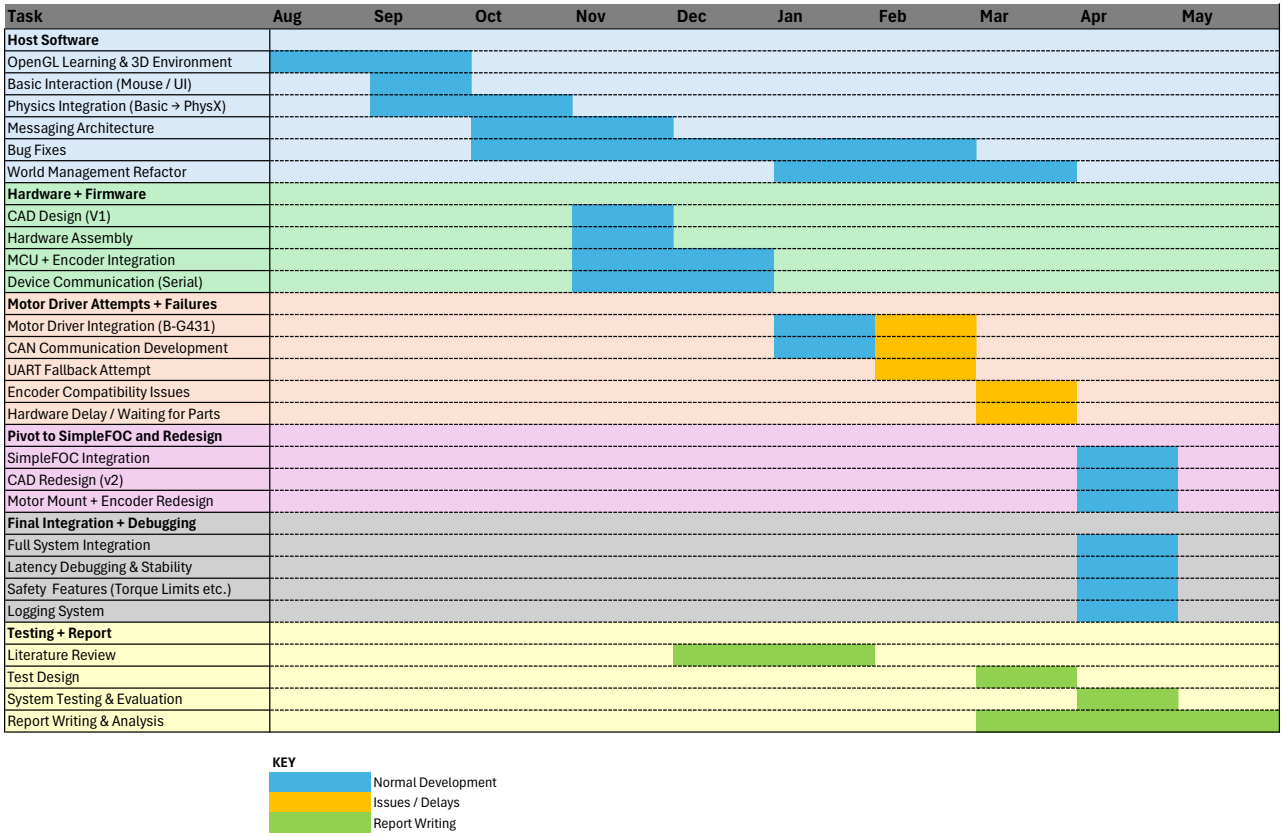


Fig. 17. Final Gantt chart showing the execution of the project over time. The chart reflects the iterative development process, including overlapping software and hardware work, as well as deviations from the original plan due to motor driver integration issues, hardware delays, and subsystem integration challenges.

C Risk Register and Health & Safety Risk Assessment

This appendix contains the project risk assessment used to identify safety, technical, and delivery risks.

MANCHESTER 1824
The University of Manchester

RISK REGISTER FOR 3rd YEAR PROJECT

Project Title:	**A Real-Time Haptic Rendering System Integrating Physics-Based Simulation and OpenGL Visualisation**		Submission Date:	1/05/2026	
Student Name:	11349510				

Project Risk	Severity			Potential			Score (Severity x Potential)	Mitigation Measures
	L	M	H	L	M	H	L=1, M=2, H=3	
Electrical faults during wiring/testing (short circuit, incorrect connections)	X				X		2	Use low-voltage systems; power off before modifying wiring; check connections before powering; use insulated connectors
Injury from moving mechanical parts (rotating joints, links)		X			X		4	Limit actuator torque; avoid contact during operation; test at low power; implement watchdog shutdown
Actuator overheating during prolonged operation		X		X			2	Monitor device temperature; limit runtime; allow cooling periods; avoid sustained high torque
Mechanical failure (belt slip, loose components)		X			X		4	Secure all components; inspect before testing; avoid excessive loading; operate within tested limits
Control instability causing unexpected motion or oscillations			X		X		6	Use conservative control gains; include damping; implement torque limits and rate limiting; test incrementally
Communication failure (loss of command stream)		X			X		4	Implement watchdog timeout to zero torque; validate communication before operation
Software bugs causing incorrect behaviour		X			X		4	Incremental testing; logging and debugging; isolate subsystems during development
Component failure (e.g. actuator failure during project)		X			X		4	Design for partial operation; maintain flexibility in testing; prioritise core functionality
Use of tools during assembly (cuts, slips)	X				X		2	Use appropriate tools; follow workshop safety procedures; maintain tidy workspace
Extended computer use (eye strain, fatigue)	X					X	3	Take regular breaks; maintain ergonomic posture; adjust screen setup

Fig. 18. Project Risk Assessment

D CPD Log

This appendix records the continuing professional development activities undertaken during the project.

Continuing Professional Development Log

Name: *Tyler Ward*

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
OpenGL and Real-Time Rendering	Self-taught OpenGL, shaders, and 3D rendering concepts using online resources.	Aug 2025 – Oct 2025	~20 hours
Haptic Rendering Methods	Read academic papers on virtual coupling and proxy-based haptics to understand force rendering approaches.	Aug 2025 – Jan 2026	~20hrs
BLDC Motor Control (SimpleFOC)	Learned to use SimpleFOC library for BLDC motor control and torque-mode operation.	Jan 2026 – Mar 2026	~10 hours
PhysX Simulation Integration	Learned how to integrate and use a physics engine (PhysX) for real-time simulation.	Jan 2026 – Feb 2026	~10hrs
Hardware Integration and Iterative Design	Gained practical experience designing and refining a mechanical system with motors and encoders.	Nov 2025 – Mar 2026	~20hrs
Working with a Large Codebase	Gained experience developing and managing a multi-component C++ codebase, including structuring modules, debugging across subsystems, and maintaining clarity in a multi-threaded architecture.	Aug 2025 – Apr 2026	~50 hrs
Technical Report Writing and Literature Review	Developed skills in structuring a technical report and conducting a focused literature review, including synthesising multiple sources, strengthening technical claims with references, and improving clarity and precision in written communication.	Dec 2025 – May 2026	~25 hrs

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
Technical Research and Literature Skills	Further develop ability to read and interpret technical research papers, including understanding methodologies, extracting key insights, and critically evaluating approaches.	Research skills, critical analysis, technical understanding	Summer 2026
Long-Term Project Planning	Improve ability to plan and manage longer-term technical projects, including setting realistic milestones, managing uncertainty, and adapting plans as challenges arise.	Project management, planning, problem-solving	Summer 2026

Fig. 19. CPD Log documenting skills and knowledge acquired during the project.

E Mechanical Design Drawing

The mechanical design drawing documents the main link dimensions, actuator placement, and encoder arrangement used by the physical prototype.

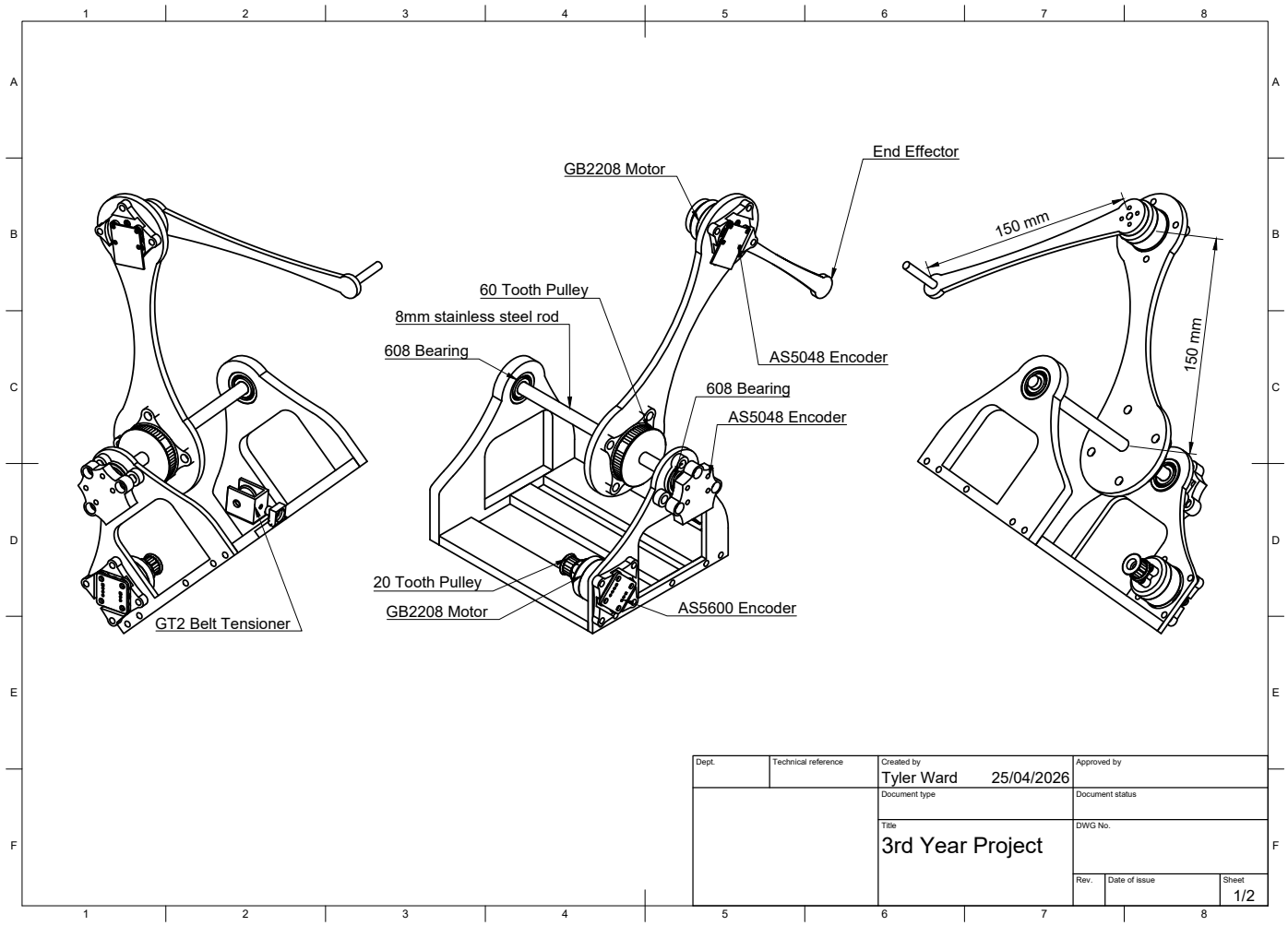


Fig. 20. CAD assembly of the 2-DOF haptic interface illustrating link geometry, actuator configuration, belt-driven base joint, and encoder placement. Link dimensions are annotated to define the kinematic model used for forward kinematics and Jacobian-based torque mapping.

F Source Code and Repository Information

Repository Link: https://github.com/TylerWardUOM/3rd-yr-project

Below is the README for the repo:

Real-Time Haptic Rendering System

Repository: TylerWardUOM/3rd-yr-project[1]

Introduction

This project implements a real-time haptic rendering system that allows a user to physically feel virtual 3D objects through a custom-built 2-DOF planar robot arm. The system tracks the position of the device's end-effector and uses that as the position of a virtual "tool" inside a simulated 3D environment. When the tool touches a virtual object, a force is calculated and fed back to the physical device — giving the user the sensation of actually pushing against something.

The software is written entirely in C++17 and runs on Windows. It brings together four real-time concerns — rendering, physics simulation, haptic force computation, and hardware communication — each running on its own thread and communicating through a shared message-bus. The rendering thread provides a live OpenGL 3D view of the scene with an ImGui control panel. The physics thread runs NVIDIA PhysX at 240 Hz for rigid-body dynamics. The haptic thread runs at 1 kHz to compute contact forces using Signed Distance Functions (SDFs). The device thread also runs at 1 kHz, reading joint angles from the robot over serial and sending torque commands back.

System Overview

```
+-----------------------------------------------------------------+
|                          Application                            |
```

[1] https://github.com/TylerWardUOM/3rd-yr-project

```
|                                                                  |
|  Main Thread      Sim Thread        Haptic Thread   Device Thread  |
|  -----------      ----------        ------------    -------------  |
|  GlSceneRenderer  WorldManager      HapticEngine    DeviceAdapter  |
|  + ImGui UI       PhysicsEnginePhysX   (1 kHz)      SerialLink     |
|  + Camera         (30 Hz / 240 Hz sub-step)          (1 kHz)       |
+------------------------------------------------------------------+
```

```
Message Bus channels:
  world.snapshots    -->  Renderer, Physics
  haptics.tool_in    -->  HapticEngine
  haptics.wrenches   -->  PhysicsEngine
  device.wrench_cmd  -->  DeviceAdapter
  haptics.snapshots  -->  Renderer (overlay)
```

Demo videos can be found in the repository `docs/Media/`.

Installation Instructions

These instructions assume a clean Windows PC with no prior project dependencies installed.

1. Install Prerequisites

| Tool | Where to get it | |—|—| | Visual Studio 2022 (with C++ Desktop workload) | https://visualstudio.microsoft.com/downloads/ | | CMake 3.20 or newer | https://cmake.org/download/ | | Git | https://git-scm.com/downloads | | vcpkg | https://github.com/microsoft/vcpkg |

Install vcpkg (package manager for C++ libraries):

```
1 git clone https://github.com/microsoft/vcpkg.git C:\Users\<you>\
    vcpkg
2 cd C:\Users\<you>\vcpkg
3 bootstrap-vcpkg.bat
```

Install PhysX via vcpkg:

```
vcpkg install physx:x64-windows
```

2. Clone the Repository

```
git clone --recurse-submodules https://github.com/TylerWardUOM/3rd-yr-project.git
cd 3rd-yr-project
```

If you already cloned without submodules, run:

```
git submodule update --init --recursive
```

This pulls in the vendored `imgui` and `glm` libraries automatically.

3. Update the vcpkg Path

Open `CMakeLists.txt` and change this line to match where your vcpkg is installed:

```
set(VCPKG_INSTALLED_DIR "C:/Users/<you>/vcpkg/installed/x64-windows")
```

4. Configure and Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

This produces two executables inside the build folder:

- `app.exe` — the main haptic rendering application
- `serial_tests.exe` — standalone serial communication test tool

5. Hardware (Optional)

To use the physical haptic device you will also need:

- A 2-DOF planar robot arm with BLDC motors and absolute magnetic encoders (AS5048 on the joints, AS5600 on the motor shafts), connected by USB serial

- Embedded firmware for the device is included in this repository under the `firmware/` folder (flashable to the target MCU). See `firmware/README` or `firmware/firmware.ino` for build/flash instructions.
- COM port access (default: COM4, 460800 baud — configurable in `src/main.cpp`)

The application will run without the hardware connected; it will fall back to mouse-based tool control.

How to Run

Main Application

```
cd build\Release
./app.exe
```

A window opens showing a 3D scene with a ground plane, a sphere, and a cube.

Controls:

| Input | Action | |—|—| | Right-click + drag | Look around (mouse-look) | | W / A / S / D | Fly camera | | Scroll wheel | Zoom | | ImGui panel (right side) | Select and move objects, change physics properties, adjust camera |

If the physical device is connected on COM4 at 460800 baud, a green sphere (device end-effector) and a blue sphere (haptic proxy) will appear in the scene as overlays.

Serial Communication Tests

The `serial_tests` executable lets you measure serial link performance without running the full simulation.

Packet rate / integrity test (checks packets received per second):

```
serial_tests rate COM4 460800 30 rate_results.csv
```

Round-trip time (RTT) test (measures latency):

```
serial_tests rtt COM4 460800 1000 2 rtt_results.csv
```

Arguments: `<port> <baud> <duration_sec or count> <interval_ms> <output_csv>`

Results are written to CSV for analysis.

Technical Details

Haptic Contact Detection — Signed Distance Functions (SDF)

Each virtual object's surface is represented by a Signed Distance Function phi(x). The value is positive outside the surface, zero on it, and negative inside. The gradient of phi points along the outward surface normal.

Plane SDF — for a plane with unit normal n and origin distance d:

```
phi(x) = n dot x - d
```

Sphere SDF — for a sphere with centre c and radius r:

```
phi(x) = |x - c| - r
```

Proxy projection — when the tool penetrates a surface (phi < 0), a haptic proxy is snapped to the nearest point on the surface:

```
x_proxy = x_tool - phi(x_tool) dot n_hat
```

This is the *god-object / proxy* method for haptic rendering (Zilles & Salisbury, 1995).

Virtual Coupling (Force Feedback)

A virtual spring-damper couples the physical tool position x_t to the constrained proxy position x_p. This matches the notation used in the report body and produces the force fed back to the device:

```
F = K(x_p - x_t) + D(v_p - v_t)
```

Parameters used in this implementation:

| Parameter | Value | Description | |—|—|—| | K | 500 N m^-1 | Spring stiffness | | M | 0.02 kg | Virtual mass (for critical damping calc) | | D | 0.7 x 2 sqrt(KM) approx. 8.9 N s m^-1 | ~70% of critical damping | | F_max | 31 N | Hard force clamp for device safety |

The equal and opposite reaction force (-F) is applied as an impulse to the corresponding PhysX rigid body each physics step, so virtual objects can be pushed around by the device.

Forward Kinematics

The physical device is a 2-DOF planar robot arm with equal link lengths L1 = L2 = 0.15 m. Joint angles q1 and q2 (in radians) are read from the encoders. The end-effector position is:

```
x = L1 cos(q1) + L2 cos(q1 + q2)
y = L1 sin(q1) + L2 sin(q1 + q2)
```

The end-effector orientation quaternion is a rotation of (q1 + q2) about the Z axis.

Jacobian Torque Computation

To render a Cartesian force F = [Fx, Fy] at the end-effector, joint torques are computed via the Jacobian transpose (tau = J^T dot F):

```
J = [ -L1 sin(q1) - L2 sin(q1+q2),   -L2 sin(q1+q2) ]
    [  L1 cos(q1) + L2 cos(q1+q2),    L2 cos(q1+q2)  ]

tau1 = J11 Fx + J21 Fy
tau2 = J12 Fx + J22 Fy
```

These torques are packed into a `TorqueCommandPacket` (with header `0xAA 0x55` and a checksum) and sent over serial to the MCU.

Threading Architecture

| Thread | Task | Rate | |—|—|—| | Main | OpenGL render loop, ImGui, camera | Display refresh | | Simulation | WorldManager step, PhysX step | ~30 Hz (1/240 s substeps) | | Haptics | SDF contact search, proxy update, force computation | 1 kHz | | Device | Serial read/parse, FK, torque send | 1 kHz |

All inter-thread communication uses lock-free message bus channels (`Channel<T>` for queues, `SnapshotChannel<T>` for latest-value reads).

Logging

On shutdown, the application writes two CSV files to the working directory:

- `device_timing.csv` — per-cycle timestamps (rx parse, tool publish, wrench consume, serial write) and signal values (q1, q2, fx, fy, tau1, tau2)
- `device_state_log.csv` — every parsed state packet from the device (sequence number, MCU timestamp, joint angles)

These are useful for diagnosing latency and communication issues.

Known Issues / Future Improvements

- **Windows-only:** `SerialLink` uses the Win32 `HANDLE` API. Porting to Linux/macOS would require replacing this with a POSIX serial implementation.
- **Hardcoded COM port:** The device port (`COM4`) and baud rate (`460800`) are hardcoded in `src/main.cpp`. These should be made configurable via a config file or command-line argument.
- **Hardcoded vcpkg path:** `CMakeLists.txt` has an absolute path to the PhysX install location. This should be replaced with a proper CMake toolchain file or `find_package` call.
- **No haptic loop stop flag:** `HapticEngine::run()` has no stop flag — the thread is detached rather than joined on shutdown. A clean shutdown flag should be added.
- **2D device only:** The current hardware is limited to a planar (XY) workspace. Extending to a 3-DOF or 6-DOF device would require updated kinematics and Jacobian.
- **No friction / texture rendering:** Only normal (penetration) forces are currently rendered. Surface friction, texture variation, and damping variation across surfaces are not yet implemented.
- **Mouse control limited:** Mouse-based tool control (fallback without the device) does not provide velocity data, so the damping term in the virtual coupling force is zero in that mode.
- **Actor rebuild is a full rebuild:** Any topology or physics property change causes all PhysX actors to be recreated from scratch. Incremental updates would be more efficient for larger scenes.